

File: aoi_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPositionManager, InspectionPosition, InspectionSequence

class CNCAOIController:
    """
    Controlador principal que integra o movimento CNC e captura de imagem
    para aplicações de Inspeção Óptica Automatizada (AOI).
    """

    def __init__(self, serial_port=None, camera_interface=None):
        """
        Inicializa o controlador AOI.

        Args:
            serial_port: Porta serial para conexão com a CNC (opcional)
            camera_interface: Interface para a câmera (opcional)
        """
        self.cnc = GRBLCNCController()
        self.camera = CameraController(camera_interface)
        self.position_manager = InspectionPositionManager()
        self.is_running_sequence = False
        self.current_sequence = None
        self.current_position_index = -1

    def connect_cnc(self, port=None, baudrate=115200):
        """Conecta à máquina CNC."""
        return self.cnc.connect(port, baudrate)

    def disconnect_cnc(self):
        """Desconecta da máquina CNC."""
        if self.cnc.is_connected:
            return self.cnc.disconnect()
        return True

    def connect_camera(self, camera_id=0):
        """Conecta à câmera."""
        return self.camera.connect(camera_id)

    def add_inspection_position(self, name, x, y, camera_params=None):
        """
        Adiciona uma posição de inspeção ao gerenciador.

        Args:
            name: Nome descritivo da posição
            x, y: Coordenadas
            camera_params: Parâmetros específicos da câmera (opcional)
        """
```

```

"""
position = InspectionPosition(name, x, y, camera_params)
self.position_manager.add_position(position)
return position

def add_current_position(self, name, camera_params=None):
    """Adiciona a posição atual como uma posição de inspeção."""
    position = self.cnc.get_current_position()
    return self.add_inspection_position(name, position['x'], position['y'], camera_params)

def create_sequence(self, name, positions=None):
    """Cria uma nova sequência de inspeção."""
    sequence = InspectionSequence(name)
    if positions:
        for pos in positions:
            sequence.add_position(pos)
    self.position_manager.add_sequence(sequence)
    return sequence

def run_sequence(self, sequence_name, callback=None):
    """
    Executa uma sequência de inspeção completa.

    Args:
        sequence_name: Nome da sequência a executar
        callback: Função chamada após cada captura com imagem e metadados
    """
    sequence = self.position_manager.get_sequence(sequence_name)
    if not sequence:
        raise ValueError(f"Sequência '{sequence_name}' não encontrada")

    self.is_running_sequence = True
    self.current_sequence = sequence
    self.current_position_index = 0

    # Iniciar execução da sequência
    return self._process_next_position(callback)

def _process_next_position(self, callback=None):
    """Processa a próxima posição na sequência."""
    if not self.is_running_sequence or self.current_position_index >= len(self.current_sequence.positions):
        self.is_running_sequence = False
        return None

    # Obtém a próxima posição
    position = self.current_sequence.positions[self.current_position_index]

    # Move para a posição
    self.cnc.move_to_absolute_position(position.x, position.y)

    # Verifica se chegou na posição

```

```

while self.cnc.is_moving():
    time.sleep(0.1)

# Captura a imagem
image = self.camera.capture(position.camera_params)

# Prepara dados para callback
result = {
    "position": position,
    "image": image,
    "timestamp": time.time(),
    "sequence": self.current_sequence.name,
    "index": self.current_position_index
}

# Avança para a próxima posição
self.current_position_index += 1

# Chama o callback se fornecido
if callback:
    callback(result)

return result

def stop_sequence(self):
    """Para a execução da sequência atual."""
    self.is_running_sequence = False

def save_positions(self, filename):
    """Salva todas as posições e sequências em um arquivo."""
    return self.position_manager.save_to_file(filename)

def load_positions(self, filename):
    """Carrega posições e sequências de um arquivo."""
    return self.position_manager.load_from_file(filename)

```

File: camera_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

class CameraController:
    """
    Classe para controlar uma câmera e capturar imagens.
    Pode ser adaptada para diferentes interfaces de câmera.
    """

```

```

def __init__(self, camera_interface=None):
    """
    Inicializa o controlador de câmera.

```

Args:

camera_interface: Interface de câmera personalizada (opcional)

"""

self.camera = camera_interface

self.is_connected = False

self.last_error = ""

def connect(self, camera_id=0):

"""

Conecta à câmera.

Args:

camera_id: ID da câmera (para OpenCV)

Returns:

bool: True se conectado com sucesso

"""

if self.camera:

Usa a interface fornecida

try:

self.is_connected = self.camera.connect()

return self.is_connected

except Exception as e:

self.last_error = str(e)

return False

else:

Implementação padrão com OpenCV

try:

import cv2

self.camera = cv2.VideoCapture(camera_id)

self.is_connected = self.camera.isOpened()

return self.is_connected

except Exception as e:

self.last_error = str(e)

return False

def disconnect(self):

"""Desconecta da câmera."""

if not self.is_connected:

return

try:

if hasattr(self.camera, 'release'):

self.camera.release()

elif hasattr(self.camera, 'disconnect'):

self.camera.disconnect()

self.is_connected = False

except Exception as e:

self.last_error = str(e)

```

def trigger_capture(self, params=None):
    """
    Aciona o trigger da câmera para captura.

    Args:
        params: Parâmetros específicos da câmera para o trigger

    Returns:
        A imagem capturada ou None em caso de erro
    """
    # Primeiro realiza o trigger
    if not self._activate_trigger(params):
        return None

    # Em seguida, captura a imagem
    return self.capture(params)

def _activate_trigger(self, params=None):
    """
    Ativa o hardware trigger da câmera, se disponível.

    Args:
        params: Parâmetros específicos para o trigger

    Returns:
        True se o trigger foi bem-sucedido, False caso contrário
    """
    if not self.is_connected:
        self.last_error = "Câmera não conectada"
        return False

    try:
        # Verifica se a interface da câmera tem um método de trigger específico
        if hasattr(self.camera, 'trigger'):
            # Interface personalizada com método de trigger
            return self.camera.trigger(params)

        # Para câmeras OpenCV padrão, criar um atraso simulando trigger por software
        # e garantir que o frame seja descartado/limpo antes da captura real
        import time
        if hasattr(self.camera, 'grab'):
            # Limpa o buffer de frames
            self.camera.grab()
            time.sleep(0.1) # Pequeno atraso para simular trigger
            return True

        return True # Se não houver trigger específico, continua com a captura normal

    except Exception as e:
        self.last_error = f"Erro ao ativar trigger: {str(e)}"
        return False

```

```

def has_hardware_trigger(self):
    """
    Verifica se a câmera possui suporte a hardware trigger.

    Returns:
        True se suporta trigger por hardware, False caso contrário
    """
    if not self.is_connected:
        return False

    # Verifica se a interface da câmera tem capacidade explícita de trigger
    return hasattr(self.camera, 'trigger') or hasattr(self.camera, 'has_hardware_trigger')

def capture(self, params=None):
    """
    Captura uma imagem da câmera.

    Args:
        params: Parâmetros específicos da câmera (exposição, etc.)

    Returns:
        A imagem capturada ou None em caso de erro
    """
    if not self.is_connected:
        self.last_error = "Câmera não conectada"
        return None

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'capture'):
            image = self.camera.capture(params)
            if image is None:
                self.last_error = "A captura de imagem retornou None"
                return None
            return image

        # Para OpenCV
        import cv2
        ret, frame = self.camera.read()
        if ret and frame is not None:
            # Aplicar parâmetros se fornecidos
            if params:
                # Exemplo: ajustar brilho/contraste
                if 'brightness' in params:
                    frame = cv2.convertScaleAbs(frame, alpha=params['brightness'])

            # Verifica se a imagem possui dados válidos
            if frame.size == 0 or frame.shape[0] == 0 or frame.shape[1] == 0:
                self.last_error = "Imagem capturada tem dimensões inválidas"
                return None
    
```

```

        return frame
    else:
        self.last_error = "Falha ao capturar imagem (sem dados ou retorno negativo)"
        return None

except Exception as e:
    self.last_error = f"Erro na captura de imagem: {str(e)}"
    return None

def set_parameters(self, **kwargs):
    """
    Define parâmetros da câmera.

    Args:
        **kwargs: Parâmetros específicos da câmera
    """
    if not self.is_connected:
        return False

    try:
        # Para interface personalizada
        if hasattr(self.camera, 'set_parameters'):
            return self.camera.set_parameters(**kwargs)

        # Para OpenCV
        import cv2
        for key, value in kwargs.items():
            if key.lower() == 'exposure':
                self.camera.set(cv2.CAP_PROP_EXPOSURE, value)
            elif key.lower() == 'focus':
                self.camera.set(cv2.CAP_PROP_FOCUS, value)
        # Adicione outros parâmetros conforme necessário

    return True

except Exception as e:
    self.last_error = str(e)
    return False

```

File: cnc_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

import time
import serial
import serial.tools.list_ports
import threading
from grbl_streamer import GrblStreamer # Importa a biblioteca grbl-streamer

class EventEmitter:

```

"""

Sistema de eventos para comunicação assíncrona.
Permite registrar múltiplos callbacks para diferentes eventos.

"""

```
def __init__(self):
```

```
    self.callbacks = {}
```

```
    self.default_callbacks = []
```

```
def connect(self, callback, event_name=None):
```

"""

Conecta um callback a um evento específico ou como callback padrão.

Args:

callback: Função a ser chamada quando o evento ocorrer

event_name: Nome do evento (se None, será adicionado aos callbacks padrão)

"""

```
if event_name is None:
```

```
    self.default_callbacks.append(callback)
```

```
else:
```

```
    if event_name not in self.callbacks:
```

```
        self.callbacks[event_name] = []
```

```
    self.callbacks[event_name].append(callback)
```

Retorna uma função que pode ser usada para desconectar este callback

```
return lambda: self.disconnect(callback, event_name)
```

```
def disconnect(self, callback, event_name=None):
```

"""

Desconecta um callback específico.

Args:

callback: Callback a ser removido

event_name: Nome do evento do qual remover (se None, remove dos callbacks padrão)

"""

```
if event_name is None:
```

```
    if callback in self.default_callbacks:
```

```
        self.default_callbacks.remove(callback)
```

```
else:
```

```
    if event_name in self.callbacks and callback in self.callbacks[event_name]:
```

```
        self.callbacks[event_name].remove(callback)
```

```
def disconnect_all(self, event_name=None):
```

"""

Desconecta todos os callbacks de um evento específico ou todos os eventos.

Args:

event_name: Nome do evento para limpar (se None, limpa todos os eventos)

"""

```
if event_name is None:
```

```
    self.callbacks.clear()
```

```
    self.default_callbacks.clear()
```

```

else:
    if event_name in self.callbacks:
        self.callbacks[event_name].clear()

```

```

def emit(self, *args, event_name=None, **kwargs):
    """

```

Emite um evento, chamando todos os callbacks registrados.

Args:

*args: Argumentos posicionais para passar aos callbacks
 event_name: Nome do evento a ser emitido (se None, apenas callbacks padrão são chamados)
 **kwargs: Argumentos nomeados para passar aos callbacks

```

    """

```

Chama todos os callbacks padrão

```

for callback in self.default_callbacks:

```

```

    try:
        callback(*args, **kwargs)
    except Exception as e:
        print(f"Erro em callback para evento: {e}")

```

Se um nome de evento for especificado, chama esses callbacks também

```

if event_name is not None and event_name in self.callbacks:

```

```

    for callback in self.callbacks[event_name]:
        try:
            callback(*args, **kwargs)
        except Exception as e:
            print(f"Erro em callback para evento '{event_name}': {e}")

```

```

def has_listeners(self, event_name=None):
    """

```

Verifica se há callbacks registrados para um evento específico ou para qualquer evento.

Args:

event_name: Nome do evento a verificar (se None, verifica todos)

Returns:

bool: True se houver callbacks registrados, False caso contrário

```

    """

```

```

if event_name is None:

```

```

    return bool(self.default_callbacks) or any(bool(callbacks) for callbacks in self.callbacks.values())

```

```

else:

```

```

    return event_name in self.callbacks and bool(self.callbacks[event_name])

```

```

class GRBLCNCController:
    """

```

Classe para controlar uma máquina CNC usando GRBL.

Implementada usando a biblioteca grbl-streamer.

```

    """

```

```

def __init__(self):

```

```

    """Inicializa o controlador CNC."""

```

```
self.grbl = None
self.lock = threading.Lock()
self.is_connected = False
self.current_position = {'x': 0, 'y': 0, 'z': 0}
self.machine_status = "Disconnected"
self.last_response = ""
self.last_error = ""
```

```
# Eventos para comunicação assíncrona
self.response_received = EventEmitter()
self.status_update = EventEmitter()
self.position_update = EventEmitter()
self.error_message = EventEmitter()
```

```
# Thread para consulta periódica de status
self.status_thread = None
self.running = False
```

```
# Controle de jog contínuo
self.jogging = False
self.current_jog_command = None
```

```
def connect(self, port=None, baudrate=115200):
```

```
    """
```

```
    Conecta à máquina CNC.
```

```
    Args:
```

```
        port: Porta serial (None para autodetecção)
```

```
        baudrate: Taxa de transmissão
```

```
    Returns:
```

```
        bool: True se conectado com sucesso
```

```
    """
```

```
    try:
```

```
        # Autodetectar porta se não especificada
```

```
        if port is None:
```

```
            ports = [p.device for p in serial.tools.list_ports.comports()]
```

```
            if not ports:
```

```
                self.last_error = "Nenhuma porta serial encontrada"
```

```
                return False
```

```
            port = ports[0]
```

```
        # Inicializa e configura o GrblStreamer
```

```
        self.grbl = GrblStreamer(port=port, baud=baudrate)
```

```
        # Configura callbacks para eventos do GrblStreamer
```

```
        self._setup_callbacks()
```

```
        # Desbloqueia a máquina
```

```
        self.send_command("$X", priority=True)
```

```

# Inicia thread de consulta de status
self.running = True
self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
self.status_thread.start()

self.is_connected = True
self.machine_status = "Idle" # Estado inicial presumido
return True

except Exception as e:
    self.last_error = str(e)
    return False

def _setup_callbacks(self):
    """
    Configura callbacks para eventos do GrblStreamer.
    Esta função deve ser adaptada com base na API real da biblioteca.
    """
    # Configuração para receber atualizações de status da máquina
    self.grbl.register_status_callback(self._on_status_update)

    # Configuração para receber respostas de comandos
    self.grbl.register_response_callback(self._on_response_received)

    # Configuração para receber notificações de erro
    self.grbl.register_error_callback(self._on_error_message)

def _status_polling_thread(self):
    """Thread para consulta periódica de status."""
    while self.running and self.is_connected:
        try:
            # Requisição de status ao GRBL a cada 200ms
            if self.grbl:
                self.grbl.get_status()
            time.sleep(0.2)
        except Exception:
            time.sleep(0.5) # Maior intervalo em caso de erro

def disconnect(self):
    """Desconecta da máquina CNC."""
    # Garante que o jog é parado antes de desconectar
    if self.jogging:
        self.stop_continuous_jog()

    self.running = False
    if self.status_thread:
        self.status_thread.join(1.0) # Espera até 1 segundo pela thread

    if self.grbl:
        try:
            self.grbl.close()

```

```
self.grbl = None
except Exception as e:
    self.last_error = str(e)
```

```
self.is_connected = False
self.machine_status = "Disconnected"
```

```
def is_moving(self):
    """Verifica se a máquina está em movimento."""
    return self.machine_status == "Run" or self.machine_status == "Jog"
```

```
def wait_for_idle(self, timeout=30):
    """
    Aguarda até que a máquina esteja ociosa.
```

Args:

timeout: Tempo máximo de espera em segundos

Returns:

bool: True se ficou ocioso, False se atingiu timeout

"""

```
start_time = time.time()
while self.is_moving():
    if time.time() - start_time > timeout:
        return False
    time.sleep(0.1)
return True
```

```
def get_current_position(self):
    """Retorna a posição atual da máquina."""
    return self.current_position.copy()
```

```
def move_to_absolute_position(self, x=None, y=None, feed_rate=1000):
    """
    Move para uma posição absoluta.
```

Args:

x, y: Coordenadas de destino (None para não mover esse eixo)

feed_rate: Velocidade de avanço em mm/min

"""

```
if not self.is_connected:
    return False
```

```
# Muda para modo absoluto
self.send_command("G90", priority=True)
```

```
# Constrói o comando
command = "G1"
if x is not None:
    command += f" X{x}"
if y is not None:
```

```

        command += f" Y{y}"
        command += f" F{feed_rate}"

    # Envia o comando
    self.send_command(command)
    return True

def move_relative(self, x=0, y=0, feed_rate=1000):
    """
    Move em relação à posição atual.

    Args:
        x, y: Distância a mover em cada eixo
        feed_rate: Velocidade de avanço em mm/min
    """
    if not self.is_connected:
        return False

    # Muda para modo relativo
    self.send_command("G91", priority=True)

    # Constrói o comando
    command = f"G1 X{x} Y{y} F{feed_rate}"

    # Envia o comando
    self.send_command(command)
    return True

def start_continuous_jog(self, axis, direction, feed_rate=1000):
    """
    Inicia um movimento jog contínuo em um eixo específico.
    O movimento continua até que stop_continuous_jog seja chamado.

    Args:
        axis: Eixo a mover ('X' ou 'Y')
        direction: Direção do movimento (1 ou -1)
        feed_rate: Velocidade em mm/min

    Returns:
        bool: True se o comando foi iniciado com sucesso
    """
    if not self.is_connected or self.jogging:
        return False

    # Para qualquer movimento anterior e garante modo relativo
    self.emergency_stop()
    time.sleep(0.1) # Pequena pausa para garantir que o GRBL processou o comando de parada
    self.send_command("~", priority=True) # Retoma após parada
    self.send_command("G91", priority=True) # Modo relativo

    # Em GRBL 1.1, podemos usar o comando de jog para movimento contínuo

```

```
# A sintaxe é: $J=G91 X[dist] Y[dist] F[feed]
distance = 1000 * direction # Distância grande para simular movimento contínuo
```

```
if axis.upper() == 'X':
    jog_command = f"$J=G91 X{distance} F{feed_rate}"
else: # assume Y
    jog_command = f"$J=G91 Y{distance} F{feed_rate}"
```

```
self.jogging = True
self.current_jog_command = jog_command
```

```
# Envia comando de jog com máxima prioridade
success = self.send_command(jog_command, priority=True)
```

```
return success
```

```
def stop_continuous_jog(self):
```

```
    """
```

```
    Para o movimento jog contínuo atual.
```

```
    Returns:
```

```
        bool: True se o comando foi enviado com sucesso
```

```
    """
```

```
    if not self.is_connected or not self.jogging:
```

```
        return False
```

```
    # O comando para cancelar jog no GRBL é o feed cancel (!)
```

```
    success = self.send_command("!", priority=True)
```

```
    # Após um breve intervalo, envia comando de retomada para liberar a máquina
```

```
    time.sleep(0.1)
```

```
    self.send_command("~", priority=True)
```

```
    self.jogging = False
```

```
    self.current_jog_command = None
```

```
    return success
```

```
def home(self):
```

```
    """Envia a máquina para a posição home."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
    self.send_command("$H")
```

```
    return True
```

```
def set_zero(self):
```

```
    """Define a posição atual como zero."""
```

```
    if not self.is_connected:
```

```
        return False
```

```
self.send_command("G92 X0 Y0")
return True
```

```
def unlock(self):
    """Desbloqueia a máquina."""
    if not self.is_connected:
        return False
```

```
self.send_command("$X", priority=True)
return True
```

```
def emergency_stop(self):
    """Para todos os movimentos imediatamente."""
    if not self.is_connected:
        return False
```

```
self.grbl.send_realtime_command("!")
self.jogging = False # Atualiza o estado de jog
return True
```

```
def send_command(self, command, priority=False):
    """
    Envia um comando personalizado ao GRBL.
```

```
Args:
    command: Comando G-code
    priority: True para enviar com prioridade
    """
```

```
if not self.is_connected or not self.grbl:
    return False
```

```
try:
    # Comandos de controle realtime (!, ~, ?) são tratados especialmente
    if command in ["!", "~", "?"]:
        self.grbl.send_realtime_command(command)
    # Comandos prioritários são enviados com prioridade
    elif priority:
        self.grbl.send_priority_command(command)
    # Comandos normais
    else:
        self.grbl.send_gcode(command)
    return True
except Exception as e:
    self.last_error = str(e)
    self.error_message.emit(str(e))
    return False
```

```
def _on_response_received(self, response):
    """Callback quando uma resposta é recebida."""
    self.last_response = response
    self.response_received.emit(response)
```

```

def _on_status_update(self, status_data):
    """
    Callback quando o status da máquina é atualizado.

    Args:
        status_data: Dados de status do GrblStreamer
    """
    # Processa os dados de status recebidos do GrblStreamer
    # A estrutura exata depende da implementação da biblioteca

    # Extrai o estado da máquina do status
    state = status_data.get('state', 'Unknown')
    self.machine_status = state

    # Extrai a posição
    position = status_data.get('position', {})
    if position:
        self.current_position = {
            'x': position.get('x', 0),
            'y': position.get('y', 0),
            'z': position.get('z', 0)
        }
        self.position_update.emit(self.current_position)

    # Formata os dados para manter compatibilidade com código existente
    status_string = f"<{state}|MPos:{self.current_position['x']:.3f},{self.current_position['y']:.3f},{self.current_position['z']:.3f}>"
    self.status_update.emit(status_string)

def _on_error_message(self, error):
    """Callback quando ocorre um erro."""
    self.last_error = error
    self.error_message.emit(error)

```

File: position_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
```

```
class InspectionPosition:
```

```
    """Representa uma posição de inspeção."""
```

```
    def __init__(self, name, x, y, camera_params=None):
```

```
        self.name = name
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.camera_params = camera_params or {}
```

```
        self.creation_time = time.time()
```

```
    def to_dict(self):
```

```
"""Converte para dicionário para serialização."""
```

```
return {  
    'name': self.name,  
    'x': self.x,  
    'y': self.y,  
    'camera_params': self.camera_params,  
    'creation_time': self.creation_time  
}
```

```
@classmethod
```

```
def from_dict(cls, data):
```

```
    """Cria a partir de um dicionário."""
```

```
    position = cls(data['name'], data['x'], data['y'], data['camera_params'])
```

```
    position.creation_time = data.get('creation_time', time.time())
```

```
    return position
```

```
class InspectionSequence:
```

```
    """Representa uma sequência de posições de inspeção."""
```

```
def __init__(self, name):
```

```
    self.name = name
```

```
    self.positions = []
```

```
    self.creation_time = time.time()
```

```
    self.last_run_time = None
```

```
def add_position(self, position):
```

```
    """Adiciona uma posição à sequência."""
```

```
    self.positions.append(position)
```

```
def remove_position(self, index):
```

```
    """Remove uma posição pelo índice."""
```

```
    if 0 <= index < len(self.positions):
```

```
        del self.positions[index]
```

```
        return True
```

```
    return False
```

```
def move_position(self, old_index, new_index):
```

```
    """Move uma posição na sequência."""
```

```
    if 0 <= old_index < len(self.positions) and 0 <= new_index < len(self.positions):
```

```
        position = self.positions.pop(old_index)
```

```
        self.positions.insert(new_index, position)
```

```
        return True
```

```
    return False
```

```
def to_dict(self):
```

```
    """Converte para dicionário para serialização."""
```

```
    return {
```

```
        'name': self.name,
```

```
        'positions': [p.to_dict() for p in self.positions],
```

```
        'creation_time': self.creation_time,
```

```
        'last_run_time': self.last_run_time
    }
```

```
@classmethod
```

```
def from_dict(cls, data, positions_dict=None):
```

```
    """Cria a partir de um dicionário."""
```

```
    sequence = cls(data['name'])
```

```
    sequence.creation_time = data.get('creation_time', time.time())
```

```
    sequence.last_run_time = data.get('last_run_time')
```

```
    # Reconstruir posições
```

```
    for pos_data in data['positions']:
```

```
        if positions_dict and pos_data['name'] in positions_dict:
```

```
            # Usa posição existente
```

```
            sequence.positions.append(positions_dict[pos_data['name']])
```

```
        else:
```

```
            # Cria nova posição
```

```
            position = InspectionPosition.from_dict(pos_data)
```

```
            sequence.positions.append(position)
```

```
    return sequence
```

```
class InspectionPositionManager:
```

```
    """Gerencia posições e sequências de inspeção."""
```

```
    def __init__(self):
```

```
        self.positions = {} # name -> position
```

```
        self.sequences = {} # name -> sequence
```

```
    def add_position(self, position):
```

```
        """Adiciona uma posição."""
```

```
        self.positions[position.name] = position
```

```
    def remove_position(self, position_name):
```

```
        """Remove uma posição pelo nome."""
```

```
        if position_name in self.positions:
```

```
            del self.positions[position_name]
```

```
        # Atualiza sequências que usam esta posição
```

```
        for sequence in self.sequences.values():
```

```
            sequence.positions = [p for p in sequence.positions if p.name != position_name]
```

```
        return True
```

```
        return False
```

```
    def get_position(self, position_name):
```

```
        """Obtém uma posição pelo nome."""
```

```
        return self.positions.get(position_name)
```

```
    def add_sequence(self, sequence):
```

```

"""Adiciona uma sequência."""
self.sequences[sequence.name] = sequence

def remove_sequence(self, sequence_name):
    """Remove uma sequência pelo nome."""
    if sequence_name in self.sequences:
        del self.sequences[sequence_name]
        return True
    return False

def get_sequence(self, sequence_name):
    """Obtém uma sequência pelo nome."""
    return self.sequences.get(sequence_name)

def save_to_file(self, filename):
    """Salva todas as posições e sequências em um arquivo JSON."""
    data = {
        'positions': {name: pos.to_dict() for name, pos in self.positions.items()},
        'sequences': {name: seq.to_dict() for name, seq in self.sequences.items()}
    }

    try:
        with open(filename, 'w') as f:
            import json
            json.dump(data, f, indent=2)
        return True
    except Exception as e:
        print(f"Erro ao salvar dados: {e}")
        return False

def load_from_file(self, filename):
    """Carrega posições e sequências de um arquivo JSON."""
    try:
        with open(filename, 'r') as f:
            import json
            data = json.load(f)

        # Limpa dados existentes
        self.positions.clear()
        self.sequences.clear()

        # Carrega posições
        for name, pos_data in data.get('positions', {}).items():
            position = InspectionPosition.from_dict(pos_data)
            self.positions[name] = position

        # Carrega sequências
        for name, seq_data in data.get('sequences', {}).items():
            sequence = InspectionSequence.from_dict(seq_data, self.positions)
            self.sequences[name] = sequence

```

```
return True
```

```
except Exception as e:
```

```
    print(f"Erro ao carregar dados: {e}")
```

```
    return False
```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
# Exporta as classes principais para facilitar importação
from .aoi_controller import CNCAOIController
from .cnc_controller import GRBLCNCCController
from .camera_controller import CameraController
from .position_manager import InspectionPosition, InspectionSequence

__version__ = '0.1.0'
```

File: config_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: consumo_lib.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```
import sys
import cv2
import time
import numpy as np

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLabel, QGroupBox, QGridLayout, QLineEdit,
                             QComboBox, QListWidget, QListWidgetItem, QFileDialog, QMessageBox, QTabWidget,
                             QSplitter, QFrame, QTableWidget, QTableWidgetItem, QHeaderView)

from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QPixmap, QImage, QFont

from aoi_lib import CNCAOIController, InspectionPosition

class ImageViewerWidget(QWidget):
    """Widget para exibir imagens capturadas pela câmera"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)
```

```
# Label para exibir a imagemgit
self.image_label = QLabel()
self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.image_label.setText("Nenhuma imagem capturada")
self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
self.image_label.setMinimumSize(400, 300)
```

```
# Informações da imagem
self.info_label = QLabel("Informações da imagem:")
```

```
self.layout.addWidget(self.image_label)
self.layout.addWidget(self.info_label)
```

```
def display_image(self, image, info_text=None):
```

```
    """Mostra uma imagem no widget"""
```

```
    if image is None:
```

```
        self.image_label.setText("Imagem inválida")
```

```
        return
```

```
# Converte imagem OpenCV para QPixmap
```

```
h, w, c = image.shape
```

```
bytes_per_line = 3 * w
```

```
q_img = QImage(image.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
```

```
pixmap = QPixmap.fromImage(q_img)
```

```
# Redimensiona se for muito grande
```

```
if pixmap.width() > 800 or pixmap.height() > 600:
```

```
    pixmap = pixmap.scaled(800, 600, Qt.AspectRatioMode.KeepAspectRatio)
```

```
self.image_label.setPixmap(pixmap)
```

```
# Atualiza informações
```

```
if info_text:
```

```
    self.info_label.setText(info_text)
```

```
else:
```

```
    self.info_label.setText(f"Imagem: {w}x{h}px")
```

```
class PositionListWidget(QWidget):
```

```
    """Widget para gerenciar lista de posições"""
```

```
    position_selected = pyqtSignal(object) # Emite a posição selecionada
```

```
def __init__(self, parent=None):
```

```
    super().__init__(parent)
```

```
    self.layout = QVBoxLayout(self)
```

```
# Título
```

```
title_label = QLabel("Posições de Inspeção")
```

```
title_font = QFont()
```

```
title_font.setBold(True)
```

```
title_label.setFont(title_font)
```

```

# Lista de posições
self.positions_list = QListWidget()
self.positions_list.currentItemChanged.connect(self.on_position_selected)

# Botões
buttons_layout = QHBoxLayout()
self.add_position_btn = QPushButton("Adicionar Posição Atual")
self.remove_position_btn = QPushButton("Remover")
buttons_layout.addWidget(self.add_position_btn)
buttons_layout.addWidget(self.remove_position_btn)

self.layout.addWidget(title_label)
self.layout.addWidget(self.positions_list)
self.layout.addLayout(buttons_layout)

# Dicionário para mapear itens da lista para objetos de posição
self.positions = {}

def add_position(self, position):
    """Adiciona uma posição à lista"""
    item_text = f"{{position.name}} ({{position.x:.2f}}, {{position.y:.2f}})"
    item = QListWidgetItem(item_text)
    self.positions_list.addItem(item)
    self.positions[item] = position

def remove_selected_position(self):
    """Remove a posição selecionada"""
    current_item = self.positions_list.currentItem()
    if current_item:
        position = self.positions.pop(current_item)
        row = self.positions_list.row(current_item)
        self.positions_list.takeItem(row)
        return position
    return None

def clear_positions(self):
    """Limpa todas as posições"""
    self.positions_list.clear()
    self.positions = {}

def get_all_positions(self):
    """Retorna todas as posições"""
    return list(self.positions.values())

def on_position_selected(self, current, previous):
    """Manipula evento de seleção de posição"""
    if current and current in self.positions:
        self.position_selected.emit(self.positions[current])

class SequenceControlWidget(QWidget):

```

```

"""Widget para controlar a execução da sequência"""
def __init__(self, parent=None):
    super().__init__(parent)
    self.layout = QVBoxLayout(self)

    # Grupo de sequência
    sequence_group = QGroupBox("Controle de Sequência")
    sequence_layout = QGridLayout()

    # Nome da sequência
    sequence_layout.addWidget(QLabel("Nome:"), 0, 0)
    self.sequence_name = QLineEdit("Sequência PCB")
    sequence_layout.addWidget(self.sequence_name, 0, 1)

    # Botões de controle
    self.create_sequence_btn = QPushButton("Criar Sequência")
    self.run_sequence_btn = QPushButton("Executar Sequência")
    self.stop_sequence_btn = QPushButton("Parar")
    self.stop_sequence_btn.setEnabled(False)

    sequence_layout.addWidget(self.create_sequence_btn, 1, 0)
    sequence_layout.addWidget(self.run_sequence_btn, 1, 1)
    sequence_layout.addWidget(self.stop_sequence_btn, 2, 0, 1, 2)

    # Status
    sequence_layout.addWidget(QLabel("Status:"), 3, 0)
    self.sequence_status = QLabel("Pronto")
    sequence_layout.addWidget(self.sequence_status, 3, 1)

    sequence_group.setLayout(sequence_layout)

    # Grupo de arquivo
    file_group = QGroupBox("Salvar/Carregar")
    file_layout = QVBoxLayout()

    self.save_btn = QPushButton("Salvar Programa")
    self.load_btn = QPushButton("Carregar Programa")

    file_layout.addWidget(self.save_btn)
    file_layout.addWidget(self.load_btn)

    file_group.setLayout(file_layout)

    self.layout.addWidget(sequence_group)
    self.layout.addWidget(file_group)
    self.layout.addStretch()

class AOIControllerApp(QMainWindow):
    """Aplicação principal para controle do sistema AOI"""
    def __init__(self):
        super().__init__()

```

```
self.setWindowTitle("Controle de Inspeção Óptica Automatizada")
self.setGeometry(100, 100, 1200, 800)
```

```
# Inicializa o controlador AOI
self.controller = CNCAOIController()
self.current_sequence = None
self.is_running_sequence = False
```

```
# Configuração da interface
self.setup_ui()
```

```
# Timer para atualizar a posição
self.update_timer = QTimer(self)
self.update_timer.timeout.connect(self.update_position_display)
self.update_timer.start(500) # Atualiza a cada 500ms
```

```
def setup_ui(self):
```

```
    # Widget central
    central_widget = QWidget()
    self.setCentralWidget(central_widget)
```

```
    # Layout principal
    main_layout = QVBoxLayout(central_widget)
```

```
    # Grupo de conexão
    connection_group = QGroupBox("Conexão")
    connection_layout = QGridLayout()
```

```
    # CNC Connection
    connection_layout.addWidget(QLabel("Porta CNC:"), 0, 0)
    self.cnc_port_combo = QComboBox()
    self.refresh_ports()
    connection_layout.addWidget(self.cnc_port_combo, 0, 1)
```

```
    self.connect_cnc_btn = QPushButton("Conectar CNC")
    self.connect_cnc_btn.clicked.connect(self.connect_cnc)
    connection_layout.addWidget(self.connect_cnc_btn, 0, 2)
```

```
    # Camera connection
    connection_layout.addWidget(QLabel("Câmera ID:"), 1, 0)
    self.camera_id_combo = QComboBox()
    self.camera_id_combo.addItems(["0", "1", "2", "3"])
    connection_layout.addWidget(self.camera_id_combo, 1, 1)
```

```
    self.connect_camera_btn = QPushButton("Conectar Câmera")
    self.connect_camera_btn.clicked.connect(self.connect_camera)
    connection_layout.addWidget(self.connect_camera_btn, 1, 2)
```

```
    # Refresh ports button
    self.refresh_ports_btn = QPushButton("Atualizar Portas")
    self.refresh_ports_btn.clicked.connect(self.refresh_ports)
```

```

connection_layout.addWidget(self.refresh_ports_btn, 0, 3)

# Test camera button
self.test_camera_btn = QPushButton("Testar Câmera")
self.test_camera_btn.clicked.connect(self.test_camera)
connection_layout.addWidget(self.test_camera_btn, 1, 3)

connection_group.setLayout(connection_layout)
main_layout.addWidget(connection_group)

# Splitter para dividir a interface em painéis
splitter = QSplitter(Qt.Orientation.Horizontal)

# Painel esquerdo: controles e lista de posições
left_panel = QWidget()
left_layout = QVBoxLayout(left_panel)

# Informações de posição
position_group = QGroupBox("Posição Atual")
position_layout = QGridLayout()

position_layout.addWidget(QLabel("X:"), 0, 0)
self.x_position = QLabel("0.000 mm")
position_layout.addWidget(self.x_position, 0, 1)

position_layout.addWidget(QLabel("Y:"), 1, 0)
self.y_position = QLabel("0.000 mm")
position_layout.addWidget(self.y_position, 1, 1)

position_layout.addWidget(QLabel("Status:"), 2, 0)
self.cnc_status = QLabel("Desconectado")
position_layout.addWidget(self.cnc_status, 2, 1)

position_group.setLayout(position_layout)
left_layout.addWidget(position_group)

# Widget de lista de posições
self.position_list_widget = PositionListWidget()
self.position_list_widget.add_position_btn.clicked.connect(self.add_current_position)
self.position_list_widget.remove_position_btn.clicked.connect(self.remove_position)
self.position_list_widget.position_selected.connect(self.on_position_selected)

left_layout.addWidget(self.position_list_widget)

# Controle de sequência
self.sequence_widget = SequenceControlWidget()
self.sequence_widget.create_sequence_btn.clicked.connect(self.create_sequence)
self.sequence_widget.run_sequence_btn.clicked.connect(self.run_sequence)
self.sequence_widget.stop_sequence_btn.clicked.connect(self.stop_sequence)
self.sequence_widget.save_btn.clicked.connect(self.save_program)
self.sequence_widget.load_btn.clicked.connect(self.load_program)

```

```

left_layout.addWidget(self.sequence_widget)

# Painel direito: visualizador de imagem e resultados
right_panel = QTabWidget()

# Aba de visualização de imagem
self.image_viewer = ImageViewerWidget()
right_panel.addTab(self.image_viewer, "Visualizador de Imagem")

# Aba de resultados
results_widget = QWidget()
results_layout = QVBoxLayout(results_widget)

self.results_table = QTableWidgetItem(0, 3)
self.results_table.setHorizontalHeaderLabels(["Posição", "Timestamp", "Status"])
self.results_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)

results_layout.addWidget(QLabel("Resultados da Inspeção:"))
results_layout.addWidget(self.results_table)

right_panel.addTab(results_widget, "Resultados")

# Adiciona painéis ao splitter
splitter.addWidget(left_panel)
splitter.addWidget(right_panel)
splitter.setSizes([400, 800])

main_layout.addWidget(splitter)

# Barra de status
self.statusBar().showMessage("Pronto para conectar")

def refresh_ports(self):
    """Atualiza a lista de portas seriais disponíveis"""
    import serial.tools.list_ports

    self.cnc_port_combo.clear()
    ports = [port.device for port in serial.tools.list_ports.comports()]

    if ports:
        self.cnc_port_combo.addItems(ports)

        # Verificar se COM9 está na lista e selecionar
        com9_index = self.cnc_port_combo.findText("COM9")
        if com9_index >= 0:
            self.cnc_port_combo.setCurrentIndex(com9_index)
            self.statusBar().showMessage("Porta COM9 detectada")
        else:
            self.statusBar().showMessage("Nenhuma porta serial encontrada")

```

```

def connect_cnc(self):
    """Conecta à máquina CNC"""
    if self.controller.cnc.is_connected:
        # Desconectar
        self.controller.cnc.disconnect()
        self.connect_cnc_btn.setText("Conectar CNC")
        self.cnc_status.setText("Desconectado")
        self.statusBar().showMessage("CNC desconectada")
    else:
        # Conectar
        port = self.cnc_port_combo.currentText()
        if not port:
            QMessageBox.warning(self, "Erro", "Selecione uma porta serial")
            return

        self.statusBar().showMessage(f"Conectando à CNC na porta {port}...")

        if self.controller.connect_cnc(port):
            self.connect_cnc_btn.setText("Desconectar CNC")
            self.cnc_status.setText("Conectado")
            self.statusBar().showMessage(f"CNC conectada na porta {port}")
        else:
            QMessageBox.critical(self, "Erro", f"Falha ao conectar à CNC: {self.controller.cnc.last_error}")

def connect_camera(self):
    """Conecta à câmera"""
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
        # Desconectar
        self.controller.camera.disconnect()
        self.connect_camera_btn.setText("Conectar Câmera")
        self.statusBar().showMessage("Câmera desconectada")
    else:
        # Conectar
        try:
            camera_id = int(self.camera_id_combo.currentText())

            self.statusBar().showMessage(f"Conectando à câmera ID {camera_id}...")

            if self.controller.connect_camera(camera_id):
                self.connect_camera_btn.setText("Desconectar Câmera")
                self.statusBar().showMessage(f"Câmera ID {camera_id} conectada")
            else:
                QMessageBox.critical(self, "Erro", f"Falha ao conectar à câmera: {self.controller.camera.last_error}")
        except ValueError:
            QMessageBox.warning(self, "Erro", "ID de câmera inválido")

def test_camera(self):
    """Testa a captura de imagem da câmera"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
    return

```

```

image = self.controller.camera.capture()

if image is not None:
    self.image_viewer.display_image(image, "Imagem de teste da câmera")
    self.statusBar().showMessage("Imagem capturada com sucesso")
else:
    QMessageBox.warning(self, "Erro", f"Falha ao capturar imagem: {self.controller.camera.last_error}")

def update_position_display(self):
    """Atualiza a exibição da posição atual"""
    if not self.controller.cnc.is_connected:
        return

    position = self.controller.cnc.get_current_position()
    self.x_position.setText(f'{position["x"]:.3f} mm')
    self.y_position.setText(f'{position["y"]:.3f} mm')

    # Atualiza status
    self.cnc_status.setText(self.controller.cnc.machine_status)

def add_current_position(self):
    """Adiciona a posição atual à lista"""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return

    position_name, ok = QInputDialog.getText(self, "Nova Posição", "Nome da posição:")

    if ok and position_name:
        position = self.controller.add_current_position(position_name)
        self.position_list_widget.add_position(position)
        self.statusBar().showMessage(f"Posição '{position_name}' adicionada")

def remove_position(self):
    """Remove a posição selecionada"""
    position = self.position_list_widget.remove_selected_position()

    if position:
        self.controller.position_manager.remove_position(position.name)
        self.statusBar().showMessage(f"Posição '{position.name}' removida")

def on_position_selected(self, position):
    """Manipula a seleção de uma posição"""
    # Poderia mover para esta posição, mostrar detalhes, etc.
    self.statusBar().showMessage(f"Posição selecionada: {position.name} ({position.x:.3f}, {position.y:.3f})")

def create_sequence(self):
    """Cria uma nova sequência com as posições atuais"""
    positions = self.position_list_widget.get_all_positions()

```

if not positions:

```
    QMessageBox.warning(self, "Aviso", "Adicione pelo menos uma posição")
    return
```

sequence_name = self.sequence_widget.sequence_name.text()

if not sequence_name:

```
    QMessageBox.warning(self, "Aviso", "Digite um nome para a sequência")
    return
```

self.current_sequence = self.controller.create_sequence(sequence_name, positions)

self.statusBar().showMessage(f"Sequência '{sequence_name}' criada com {len(positions)} posições")

self.sequence_widget.sequence_status.setText(f"Criada: {len(positions)} posições")

def run_sequence(self):

"""Executa a sequência atual"""

if not self.current_sequence:

```
    QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
    return
```

if not self.controller.cnc.is_connected:

```
    QMessageBox.warning(self, "Aviso", "CNC não conectada")
    return
```

if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:

```
    QMessageBox.warning(self, "Aviso", "Câmera não conectada")
    return
```

Limpa resultados anteriores

self.results_table.setRowCount(0)

Configura UI para execução

self.is_running_sequence = True

self.sequence_widget.run_sequence_btn.setEnabled(False)

self.sequence_widget.stop_sequence_btn.setEnabled(True)

self.sequence_widget.sequence_status.setText("Executando...")

Inicia a execução

try:

self.statusBar().showMessage(f"Executando sequência '{self.current_sequence.name}'...")

Nesta versão de GUI, usaremos um thread separado para executar a sequência

self.run_thread = SequenceRunnerThread(self.controller, self.current_sequence.name)

self.run_thread.image_captured.connect(self.on_image_captured)

self.run_thread.sequence_completed.connect(self.on_sequence_completed)

self.run_thread.sequence_error.connect(self.on_sequence_error)

self.run_thread.start()

except Exception as e:

self.statusBar().showMessage(f"Erro ao executar sequência: {str(e)}")

self.sequence_widget.sequence_status.setText("Erro")

self.on_sequence_completed()

```

def stop_sequence(self):
    """Para a execução da sequência atual"""
    if self.is_running_sequence:
        self.controller.stop_sequence()
        self.statusBar().showMessage("Execução de sequência interrompida")
        self.on_sequence_completed()

def on_image_captured(self, result):
    """Chamado quando uma imagem é capturada durante a execução da sequência"""
    position = result["position"]
    image = result["image"]
    timestamp = result["timestamp"]

    # Exibe a imagem
    self.image_viewer.display_image(image, f"Posição: {position.name} ({position.x:.3f}, {position.y:.3f})")

    # Adiciona ao registro de resultados
    row = self.results_table.rowCount()
    self.results_table.insertRow(row)
    self.results_table.setItem(row, 0, QTableWidgetItem(position.name))
    self.results_table.setItem(row, 1, QTableWidgetItem(time.strftime("%H:%M:%S", time.localtime(timestamp))))
    self.results_table.setItem(row, 2, QTableWidgetItem("Capturado"))

def on_sequence_completed(self):
    """Chamado quando a execução da sequência é concluída"""
    self.is_running_sequence = False
    self.sequence_widget.run_sequence_btn.setEnabled(True)
    self.sequence_widget.stop_sequence_btn.setEnabled(False)
    self.sequence_widget.sequence_status.setText("Concluída")
    self.statusBar().showMessage("Execução de sequência concluída")

def on_sequence_error(self, error_message):
    """Chamado quando ocorre um erro durante a execução da sequência"""
    QMessageBox.critical(self, "Erro na Sequência", error_message)
    self.on_sequence_completed()

def save_program(self):
    """Salva o programa de inspeção atual"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    filename, _ = QFileDialog.getSaveFileName(self, "Salvar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if not filename.endswith('.json'):
            filename += '.json'

        if self.controller.save_positions(filename):
            self.statusBar().showMessage(f"Programa salvo em {filename}")

```

else:

QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

def load_program(self):

"""Carrega um programa de inspeção salvo"""

filename, _ = QFileDialog.getOpenFileName(self, "Carregar Programa", "", "Arquivos JSON (*.json)")

if filename:

if self.controller.load_positions(filename):

Atualiza a interface

self.position_list_widget.clear_positions()

Adiciona posições carregadas à lista

for name, position in self.controller.position_manager.positions.items():

self.position_list_widget.add_position(position)

Se há sequências, usa a primeira como atual

if self.controller.position_manager.sequences:

sequence_name = next(iter(self.controller.position_manager.sequences))

self.current_sequence = self.controller.position_manager.sequences[sequence_name]

self.sequence_widget.sequence_name.setText(sequence_name)

self.sequence_widget.sequence_status.setText(f"Carregada: {len(self.current_sequence.positions)} posições")

self.statusBar().showMessage(f"Programa carregado de {filename}")

else:

QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")

def closeEvent(self, event):

"""Manipula o evento de fechamento da janela"""

Para a execução da sequência, se houver

if self.is_running_sequence:

self.controller.stop_sequence()

Desconecta da câmera e CNC

if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:

self.controller.camera.disconnect()

if self.controller.cnc.is_connected:

self.controller.cnc.disconnect()

event.accept()

class SequenceRunnerThread(QThread):

"""Thread para executar uma sequência de inspeção"""

image_captured = pyqtSignal(dict) # Emite resultados da captura

sequence_completed = pyqtSignal() # Emite quando a sequência é concluída

sequence_error = pyqtSignal(str) # Emite quando ocorre um erro

def __init__(self, controller, sequence_name):

super().__init__()

self.controller = controller

```

self.sequence_name = sequence_name

def run(self):
    try:
        # Execute a sequência e processe os resultados
        def process_result(result):
            self.image_captured.emit(result)

        self.controller.run_sequence(self.sequence_name, process_result)
        self.sequence_completed.emit()

    except Exception as e:
        self.sequence_error.emit(str(e))

# Classe QDialog que estava faltando
from PyQt6.QtWidgets import QDialog

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AOIControllerApp()
    window.show()
    sys.exit(app.exec())

```

File: controlador.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```

import sys
import serial
import serial.tools.list_ports
import time
import re
import threading
import logging
from queue import Queue, Empty, PriorityQueue
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QLabel, QSlider, QGroupBox,
                             QGridLayout, QLineEdit, QComboBox, QProgressBar,
                             QFrame, QPushButton)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer, QObject
from PyQt6.QtGui import QFont, QColor, QPalette, QIcon

# Importa a biblioteca grbl-streamer
from grbl_streamer import GrblStreamer

# Configuração de logging para diagnóstico
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("grbl_comm.log"),

```

```

        logging.StreamHandler()
    ]
)

```

```

logger = logging.getLogger("GRBLComm")

```

```

class GRBLCommunicationThread(QThread):

```

```

    """Thread para lidar com a comunicação com GRBL sem bloquear a interface"""

```

```

    response_received = pyqtSignal(str)

```

```

    status_update = pyqtSignal(str)

```

```

    position_update = pyqtSignal(dict)

```

```

    error_message = pyqtSignal(str)

```

```

    def __init__(self, parent=None):

```

```

        super().__init__(parent)

```

```

        self.serial_port = None

```

```

        self.is_connected = False

```

```

        self.running = True

```

```

        # Usando PriorityQueue como em consumo_lib.py

```

```

        self.command_queue = PriorityQueue()

```

```

        # Usando um único lock para simplicidade

```

```

        self.lock = threading.Lock()

```

```

        self.is_moving = False

```

```

        self.last_status_check = 0

```

```

        self.last_position = {'x': 0, 'y': 0, 'z': 0}

```

```

        self.machine_state = "Unknown"

```

```

        logger.info("GRBLCommunicationThread inicializada")

```

```

    def set_serial_port(self, serial_port):

```

```

        """Define a porta serial para comunicação."""

```

```

        logger.info(f"Definindo porta serial: {serial_port}")

```

```

        self.serial_port = serial_port

```

```

        self.is_connected = True

```

```

    def disconnect(self):

```

```

        """Desconecta a porta serial."""

```

```

        logger.info("Desconectando porta serial")

```

```

        self.is_connected = False

```

```

        if self.serial_port and self.serial_port.is_open:

```

```

            try:

```

```

                with self.lock:

```

```

                    logger.debug("Fechando porta serial")

```

```

                    self.serial_port.close()

```

```

                    logger.info("Porta serial fechada com sucesso")

```

```

            except Exception as e:

```

```

                logger.error(f"Erro ao fechar porta serial: {e}")

```

```

    def send_command(self, command, priority=False):

```

```

        """

```

```

        Adiciona um comando à fila de execução, com opção de prioridade.

```

Args:

command: O comando GCODE

priority: Se True, o comando tem prioridade

"""

Comandos de controle imediato (!, ~, ?) são sempre processados imediatamente

if command in ['!', '~', '?']:

 priority_level = -1 # Prioridade máxima

else:

 priority_level = 0 if priority else 1

logger.debug(f"Enviando comando: '{command}' (priority={priority_level})")

self.command_queue.put((priority_level, command))

def run(self):

 """Método principal da thread - processa os comandos e monitora o status"""

 logger.info("Thread de comunicação iniciada")

 last_diagnostic_time = 0

 last_command_time = 0

 while self.running:

 try:

 # Diagnóstico periódico

 current_time = time.time()

 if current_time - last_diagnostic_time > 10:

 if self.is_connected and self.serial_port and self.serial_port.is_open:

 buffer_size = self.serial_port.in_waiting if hasattr(self.serial_port, 'in_waiting') else 0

 logger.info(f"Diagnóstico de comunicação: Conectado, buffer_in={buffer_size}, is_moving={self.is_moving}, e

 # Limpar buffer se estiver muito grande

 if buffer_size > 200:

 logger.warning(f"Buffer muito grande ({buffer_size} bytes), limpando...")

 with self.lock:

 self.serial_port.reset_input_buffer()

 else:

 logger.info(f"Diagnóstico de comunicação: Desconectado, is_moving={self.is_moving}, estado={self.machine

 last_diagnostic_time = current_time

 # Verificar conexão

 if not self.is_connected or not self.serial_port or not self.serial_port.is_open:

 logger.debug("Não conectado, aguardando...")

 time.sleep(0.1)

 continue

 # Verificar e processar dados recebidos mesmo sem enviar comando

 if self.serial_port.in_waiting > 0:

 with self.lock:

 data = self.serial_port.readline().decode('utf-8', errors='replace').strip()

 if data:

 logger.debug(f"Dados recebidos: '{data}'")

```

# Processar dados recebidos
if data.startswith('<') and '|' in data:
    self._parse_status(data)
elif "ok" in data or "error" in data:
    self.response_received.emit(data)

# Limita a taxa de envio de comandos
can_send_command = True
if current_time - last_command_time < 0.1: # Máximo 10 comandos por segundo
    can_send_command = False

# Processa comandos na fila
if can_send_command and not self.command_queue.empty():
    priority, command = self.command_queue.get(block=False)

    # Comandos prioritários ou de controle podem sempre ser enviados
    if priority <= 0 or command in ['!', '~', '?']:
        pass # Sempre permite
    # Comandos regulares verificam o estado da máquina
    elif self.is_moving and "Hold" not in self.machine_state:
        logger.debug(f"Máquina em movimento, enfileirando comando: {command}")
        self.command_queue.put((priority, command))
        self.command_queue.task_done()
        time.sleep(0.05)
        continue

    # Preparar comando
    if not command.endswith('\n'):
        command += '\n'

    # Identificar tipo de comando
    is_movement_cmd = any(cmd in command for cmd in ['G0', 'G1', 'G2', 'G3'])

    # Enviar comando
    with self.lock:
        logger.debug(f"Enviando bytes: {command.encode()}")
        self.serial_port.write(command.encode())
        self.serial_port.flush()

    # Se for comando de movimento, atualiza estado
    if is_movement_cmd:
        logger.info("Atualizando flag is_moving para True")
        self.is_moving = True

    # Atualiza timestamp
    last_command_time = current_time

    # Para comandos rápidos (controle), não espera resposta
    if command.strip() in ['!', '~', '?']:
        self.command_queue.task_done()

```

continue

Para outros comandos, espera resposta

start_time = time.time()

response = ""

Lê respostas com timeout curto (200ms)

while (time.time() - start_time) < 0.2:

if self.serial_port.in_waiting > 0:

line = self.serial_port.readline().decode('utf-8', errors='replace').strip()

if line:

if not line.startswith('<'):

response += line + " "

Se recebeu ok ou error, finaliza

if "ok" in line or "error" in line:

break

Status atualiza estado mas não é considerado resposta final

elif line.startswith('<') and '|' in line:

self._parse_status(line)

else:

time.sleep(0.01)

Finaliza comando

self.command_queue.task_done()

Processa resposta

if response:

self.response_received.emit(response.strip())

if is_movement_cmd and "error" in response.lower():

logger.warning(f"Erro GRBL em comando de movimento: {response}")

self.error_message.emit(f"Erro GRBL: {response}")

self.is_moving = False

Verificação periódica de status

if (current_time - self.last_status_check) > 0.2:

self._check_status_simple()

self.last_status_check = current_time

Pausa pequena para não consumir CPU

time.sleep(0.01)

except Exception as e:

logger.error(f"Erro na thread de comunicação: {e}", exc_info=True)

self.error_message.emit(f"Erro na thread de comunicação: {str(e)}")

time.sleep(0.1)

def _check_status_simple(self):

"""Verifica o status atual do GRBL (implementação simplificada)"""

try:

if not self.is_connected or not self.serial_port or not self.serial_port.is_open:

return

```

# Apenas envia o comando de status
with self.lock:
    logger.debug("Enviando comando de status '?'")
    self.serial_port.write(b"?.")
    self.serial_port.flush()
    # Não aguardamos resposta aqui, ela será processada no próximo ciclo

except Exception as e:
    logger.error(f"Erro ao verificar status: {e}")

# Modifique o método _parse_status na classe GRBLCommunicationThread
def _parse_status(self, status_line):
    """
    Analisa a linha de status do GRBL.

    Formato típico: <Idle|MPos:0.000,0.000,0.000|FS:0,0|WCO:0.000,0.000,0.000>
    """
    try:
        logger.debug(f"Analisando status: '{status_line}'")

        # Extrai o estado da máquina
        if status_line.startswith('<') and '|' in status_line:
            state = status_line[1:status_line.find('|')]
            old_state = self.machine_state
            self.machine_state = state
            logger.debug(f"Estado da máquina: {old_state} -> {state}")

        # Atualiza a flag de movimento com base no estado
        old_moving = self.is_moving

        # Estados onde a máquina está definitivamente parada
        if state in ["Idle", "Alarm", "Error", "Door", "Check", "Home"]:
            self.is_moving = False
        # Estados onde a máquina está definitivamente em movimento
        elif state in ["Run", "Jog"]:
            self.is_moving = True
        # Estado Hold - a máquina está temporariamente parada, mas não completamente Idle
        elif state.startswith("Hold"):
            # Para os nossos propósitos, consideramos Hold como "não está em movimento"
            # Isso permite que novos comandos sejam enviados após liberar o Hold
            self.is_moving = False

        if old_moving != self.is_moving:
            logger.info(f"Flag is_moving atualizada: {old_moving} -> {self.is_moving}")

        # Emite estado para atualizar UI quando o estado muda
        if old_state != state:
            self.status_update.emit(status_line)

    # Extrai a posição

```

```
match = re.search(r'(?:(MPos|WPos):(-?\d+\.\d+),(-?\d+\.\d+),(-?\d+\.\d+))', status_line)
```

```
if match:
```

```
    x, y, z = match.groups()
```

```
    position = {
```

```
        'x': float(x),
```

```
        'y': float(y),
```

```
        'z': float(z)
```

```
    }
```

```
    # Emite atualização apenas se a posição mudou significativamente (> 0.001mm)
```

```
    if (abs(position['x'] - self.last_position['x']) > 0.001 or
```

```
        abs(position['y'] - self.last_position['y']) > 0.001 or
```

```
        abs(position['z'] - self.last_position['z']) > 0.001):
```

```
        self.position_update.emit(position)
```

```
        self.last_position = position
```

```
except Exception as e:
```

```
    logger.error(f"Erro ao analisar status: {e}", exc_info=True)
```

```
class SignalEmitter(QObject):
```

```
    """Classe para emitir sinais Qt de dentro da biblioteca grbl-streamer"""
```

```
    response_received = pyqtSignal(str)
```

```
    status_update = pyqtSignal(dict)
```

```
    error_message = pyqtSignal(str)
```

```
class DirectionalButton(QPushButton):
```

```
    """Botão direcional com setas para controle intuitivo de movimento"""
```

```
    def __init__(self, direction, parent=None):
```

```
        super().__init__(parent)
```

```
        self.direction = direction
```

```
    # Configura a aparência do botão
```

```
    self.setMinimumSize(60, 60)
```

```
    self.setMaximumSize(60, 60)
```

```
    # Define texto/símbolo com base na direção
```

```
    if direction == "up":
```

```
        self.setText("↑")
```

```
    elif direction == "down":
```

```
        self.setText("↓")
```

```
    elif direction == "left":
```

```
        self.setText("←")
```

```
    elif direction == "right":
```

```
        self.setText("→")
```

```
    # Estiliza o botão para parecer com botão de jog
```

```
    font = QFont()
```

```
    font.setBold(True)
```

```
    font.setPointSize(20)
```

```
    self.setFont(font)
```

```

class JogButton(QPushButton):
    """Botão personalizado para movimentos jog"""
    def __init__(self, text, parent=None):
        super().__init__(text, parent)

        # Estilização
        self.setMinimumSize(60, 60)
        font = QFont()
        font.setBold(True)
        font.setPointSize(12)
        self.setFont(font)

class PositionDisplay(QFrame):
    """Widget para exibir a posição atual da máquina"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self setFrameShape(QFrame.Shape.StyledPanel)
        self setFrameShadow(QFrame.Shadow.Raised)

        layout = QGridLayout(self)

        # Rótulos para cabeçalhos
        title_font = QFont()
        title_font.setBold(True)

        title_label = QLabel("Posição Atual:")
        title_label.setFont(title_font)
        layout.addWidget(title_label, 0, 0, 1, 2)

        layout.addWidget(QLabel("X:"), 1, 0)
        layout.addWidget(QLabel("Y:"), 2, 0)

        # Valores de posição
        self.x_pos = QLabel("0.000")
        self.y_pos = QLabel("0.000")

        pos_font = QFont("Monospace")
        pos_font.setPointSize(12)
        pos_font.setBold(True)

        self.x_pos.setFont(pos_font)
        self.y_pos.setFont(pos_font)

        self.x_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)
        self.y_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)

        layout.addWidget(self.x_pos, 1, 1)
        layout.addWidget(self.y_pos, 2, 1)

    def update_position(self, position):

```

```
"""Atualiza os valores de posição exibidos"""
self.x_pos.setText(f'{position.get('x', 0):.3f} mm')
self.y_pos.setText(f'{position.get('y', 0):.3f} mm')
```

```
class JogController:
```

```
    """Controlador dedicado para operações de jogging"""
```

```
    def __init__(self, comm_thread):
```

```
        self.comm_thread = comm_thread
```

```
        self.is_jogging = False
```

```
        self.current_direction = None
```

```
        self.current_axis = None
```

```
        self.jog_timer = QTimer()
```

```
        self.jog_timer.timeout.connect(self._execute_jog_step)
```

```
        self.jog_step_size = 1.0 # Aumentado para 1mm por passo para movimento mais visível
```

```
        self.jog_speed = 1000 # mm/min
```

```
        self.last_command_time = 0
```

```
        self.command_sent = False
```

```
    def start_jog(self, axis, direction):
```

```
        """Inicia o modo de jogging contínuo"""
```

```
        logger.debug(f"JogController: iniciando jog {axis}{'+' if direction > 0 else '-'}")
```

```
        # Se já estiver em jog com os mesmos parâmetros, não reinicia
```

```
        if self.is_jogging and self.current_axis == axis and self.current_direction == direction:
            return
```

```
        # Para qualquer jog anterior antes de iniciar um novo
```

```
        self.stop_jog()
```

```
        # Configura o novo movimento
```

```
        self.is_jogging = True
```

```
        self.current_axis = axis
```

```
        self.current_direction = direction
```

```
        self.command_sent = False
```

```
        # Inicia o processo de jog com base no estado atual da máquina
```

```
        if "Hold" in self.comm_thread.machine_state:
```

```
            logger.debug("JogController: máquina em Hold, enviando comando de retomada")
```

```
            self.comm_thread.send_command("~", priority=True)
```

```
            # Aguarda para dar tempo ao comando de retomada ser processado
```

```
            QTimer.singleShot(100, self._prepare_for_jog)
```

```
        else:
```

```
            self._prepare_for_jog()
```

```
    def _prepare_for_jog(self):
```

```
        """Prepara a máquina para realizar um movimento de jog"""
```

```
        # Garante modo relativo
```

```
        self.comm_thread.send_command("G91", priority=True)
```

```
        # Espera um curto período para garantir que o comando seja processado
```

```
QTimer.singleShot(50, self._start_jog_timer)
```

```
def _start_jog_timer(self):
```

```
    """Inicia o timer que controla o movimento de jog"""
```

```
    if not self.is_jogging:
```

```
        return # Verifica se o jog ainda está ativo
```

```
    # Executa o primeiro movimento imediatamente
```

```
    self._execute_jog_step()
```

```
    # Configura o timer para repetir o movimento periodicamente
```

```
    # Intervalo maior para melhor estabilidade
```

```
    self.jog_timer.start(300) # 300ms entre passos
```

```
def _execute_jog_step(self):
```

```
    """Executa um único passo de jog"""
```

```
    if not self.is_jogging:
```

```
        return
```

```
    # Impede o envio de comandos muito frequentes
```

```
    current_time = time.time()
```

```
    if current_time - self.last_command_time < 0.2:
```

```
        return
```

```
    # Verifica condições de estado da máquina
```

```
    if "Hold" in self.comm_thread.machine_state:
```

```
        logger.debug("JogController: detectado estado Hold durante movimento, enviando retomada")
```

```
        self.comm_thread.send_command("~", priority=True)
```

```
        return
```

```
    # Calcula a distância do movimento
```

```
    distance = self.current_direction * self.jog_step_size
```

```
    # Cria o comando de movimento
```

```
    if self.current_axis == "X":
```

```
        command = f"G0 X{distance} F{self.jog_speed}"
```

```
    else: # assume "Y"
```

```
        command = f"G0 Y{distance} F{self.jog_speed}"
```

```
    logger.debug(f"JogController: enviando comando {command}")
```

```
    # Envia o comando com alta prioridade
```

```
    self.comm_thread.send_command(command, priority=True)
```

```
    self.last_command_time = current_time
```

```
    self.command_sent = True
```

```
def stop_jog(self):
```

```
    """Para o movimento jog"""
```

```
    if not self.is_jogging:
```

```
        return
```

```
logger.debug("JogController: parando jog")
```

```
# Interrompe o timer
```

```
self.jog_timer.stop()
```

```
self.is_jogging = False
```

```
# Envia comando de parada apenas se um comando foi enviado antes
```

```
if self.command_sent:
```

```
    # Envia feed hold para parar movimento
```

```
    self.comm_thread.send_command("!", priority=True)
```

```
# Após uma pequena pausa, envia comando de retomada para liberar máquina
```

```
QTimer.singleShot(150, self._send_resume_after_stop)
```

```
def _send_resume_after_stop(self):
```

```
    """Envia comando de retomada após parada para liberar a máquina"""
```

```
# Envio do comando "~" para liberar máquina do estado Hold
```

```
self.comm_thread.send_command("~", priority=True)
```

```
# Solicita status atualizado
```

```
QTimer.singleShot(50, lambda: self.comm_thread.send_command("?", priority=True))
```

```
class GRBLController(QMainWindow):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.setWindowTitle("Controle CNC GRBL Avançado")
```

```
        self.setGeometry(100, 100, 1000, 700)
```

```
# Inicialização do sistema de comunicação
```

```
self.signals = SignalEmitter()
```

```
self.grbl = None
```

```
self.is_connected = False
```

```
self.current_position = {'x': 0, 'y': 0, 'z': 0}
```

```
self.machine_status = "Desconectado"
```

```
self.last_command = ""
```

```
self.last_response = ""
```

```
self.last_error = ""
```

```
# Controle de jog contínuo
```

```
self.jogging = False
```

```
self.current_jog_axis = None
```

```
self.current_jog_direction = None
```

```
# Estado da máquina
```

```
self.current_motion_mode = "G91" # Começa em modo relativo
```

```
# Conectar os sinais
```

```
self.signals.response_received.connect(self.on_response_received)
```

```
self.signals.status_update.connect(self.on_status_update)
```

```
self.signals.error_message.connect(self.on_error_message)
```

```

# Configuração da interface
self.init_ui()

# Timer para atualizar o status da interface
self.ui_timer = QTimer(self)
self.ui_timer.timeout.connect(self.update_ui_state)
self.ui_timer.start(300) # Atualiza a cada 300ms

def init_ui(self):
    # Widget central
    central_widget = QWidget()
    self.setCentralWidget(central_widget)

    # Layout principal
    main_layout = QVBoxLayout(central_widget)

    # Grupo de conexão
    connection_group = QGroupBox("Conexão Arduino")
    connection_layout = QHBoxLayout()

    self.port_combo = QComboBox()

    self.connect_button = QPushButton("Conectar")
    self.connect_button.clicked.connect(self.toggle_connection)

    self.refresh_button = QPushButton("Atualizar Portas")
    self.refresh_button.clicked.connect(self.refresh_ports)

    connection_layout.addWidget(QLabel("Porta:"))
    connection_layout.addWidget(self.port_combo)
    connection_layout.addWidget(self.connect_button)
    connection_layout.addWidget(self.refresh_button)
    connection_group.setLayout(connection_layout)

    main_layout.addWidget(connection_group)

    # Layout horizontal para controles e status
    control_status_layout = QHBoxLayout()

    # Grupo de controle dos eixos (lado esquerdo)
    axis_group = QGroupBox("Controle de Eixos")
    axis_layout = QGridLayout()

    # Controles do eixo X
    axis_layout.addWidget(QLabel("Eixo X:"), 0, 0, Qt.AlignmentFlag.AlignRight)

    x_control_layout = QHBoxLayout()
    self.x_minus_button = JogButton("X-")
    self.x_minus_button.pressed.connect(lambda: self.start_jog("X", -1))
    self.x_minus_button.released.connect(self.stop_jog)

```

```

self.x_plus_button = JogButton("X+")
self.x_plus_button.pressed.connect(lambda: self.start_jog("X", 1))
self.x_plus_button.released.connect(self.stop_jog)

self.x_distance = QLineEdit("1")
self.x_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

x_control_layout.addWidget(self.x_minus_button)
x_control_layout.addWidget(self.x_distance)
x_control_layout.addWidget(self.x_plus_button)

axis_layout.addLayout(x_control_layout, 0, 1)

# Controles do eixo Y
axis_layout.addWidget(QLabel("Eixo Y:"), 1, 0, Qt.AlignmentFlag.AlignRight)

y_control_layout = QHBoxLayout()
self.y_minus_button = JogButton("Y-")
self.y_minus_button.pressed.connect(lambda: self.start_jog("Y", -1))
self.y_minus_button.released.connect(self.stop_jog)

self.y_plus_button = JogButton("Y+")
self.y_plus_button.pressed.connect(lambda: self.start_jog("Y", 1))
self.y_plus_button.released.connect(self.stop_jog)

self.y_distance = QLineEdit("1")
self.y_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

y_control_layout.addWidget(self.y_minus_button)
y_control_layout.addWidget(self.y_distance)
y_control_layout.addWidget(self.y_plus_button)

axis_layout.addLayout(y_control_layout, 1, 1)

# Controle direcional em formato de teclado numérico (8, 4, 6, 2)
directional_group = QGroupBox("Controle Direcional")
directional_layout = QGridLayout()

self.dir_up_button = DirectionalButton("up")
self.dir_left_button = DirectionalButton("left")
self.dir_right_button = DirectionalButton("right")
self.dir_down_button = DirectionalButton("down")

# Conectar eventos de pressionar/soltar para movimentos contínuos
self.dir_up_button.pressed.connect(lambda: self.start_continuous_jog("Y", 1))
self.dir_up_button.released.connect(self.stop_continuous_jog)

self.dir_down_button.pressed.connect(lambda: self.start_continuous_jog("Y", -1))
self.dir_down_button.released.connect(self.stop_continuous_jog)

```

```

self.dir_left_button.pressed.connect(lambda: self.start_continuous_jog("X", -1))
self.dir_left_button.released.connect(self.stop_continuous_jog)

self.dir_right_button.pressed.connect(lambda: self.start_continuous_jog("X", 1))
self.dir_right_button.released.connect(self.stop_continuous_jog)

# Organização em grade para parecer com teclado numérico
directional_layout.addWidget(self.dir_up_button, 0, 1)
directional_layout.addWidget(self.dir_left_button, 1, 0)
directional_layout.addWidget(self.dir_right_button, 1, 2)
directional_layout.addWidget(self.dir_down_button, 2, 1)

# Jog step - valor incremental para os botões direcionais
step_layout = QHBoxLayout()
step_layout.addWidget(QLabel("Passo:"))
self.jog_step = QLineEdit("1")
self.jog_step.setAlignment(Qt.AlignmentFlag.AlignCenter)
step_layout.addWidget(self.jog_step)
step_layout.addWidget(QLabel("mm"))

directional_layout.addLayout(step_layout, 3, 0, 1, 3)
directional_group.setLayout(directional_layout)

# Adiciona o grupo direcional ao layout principal
axis_layout.addWidget(directional_group, 2, 0, 2, 2)

# Velocidade de movimento (feed rate)
axis_layout.addWidget(QLabel("Velocidade:"), 4, 0, Qt.AlignmentFlag.AlignRight)

feed_layout = QHBoxLayout()
self.feed_rate = QLineEdit("1000")
self.feed_rate.setAlignment(Qt.AlignmentFlag.AlignCenter)

# Slider para controle de velocidade
self.feed_slider = QSlider(Qt.Orientation.Horizontal)
self.feed_slider.setMinimum(100)
self.feed_slider.setMaximum(5000)
self.feed_slider.setValue(1000)
self.feed_slider.setTickPosition(QSlider.TickPosition.TicksBelow)
self.feed_slider.setTickInterval(500)

# Conectar slider e campo de texto
self.feed_slider.valueChanged.connect(self.update_feed_rate)
self.feed_rate.textChanged.connect(self.update_feed_slider)

feed_layout.addWidget(self.feed_rate)
feed_layout.addWidget(QLabel("mm/min"))
feed_layout.addWidget(self.feed_slider)

axis_layout.addLayout(feed_layout, 4, 1)

```

```

# Modos de movimento
axis_layout.addWidget(QLabel("Modo:"), 5, 0, Qt.AlignmentFlag.AlignRight)

mode_layout = QHBoxLayout()
self.mode_absolute = QPushButton("Passo a Passo (G90)")
self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))

self.mode_relative = QPushButton("Contínuo (G91)")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Começa em modo relativo
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))

mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)

axis_layout.addLayout(mode_layout, 5, 1)

# Botões de controle da máquina
control_buttons_layout = QGridLayout()

self.home_button = QPushButton("Home ($H)")
self.home_button.clicked.connect(self.home)

self.unlock_button = QPushButton("Desbloquear ($X)")
self.unlock_button.clicked.connect(self.unlock)

self.zero_button = QPushButton("Zerar Posição (G92)")
self.zero_button.clicked.connect(self.set_zero)

self.reset_button = QPushButton("Reset Alarme")
self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold;")
self.reset_button.clicked.connect(self.reset_alarm)

self.stop_button = QPushButton("PARAR (!)")
self.stop_button.setStyleSheet("background-color: red; color: white; font-weight: bold;")
self.stop_button.clicked.connect(self.emergency_stop)

control_buttons_layout.addWidget(self.home_button, 0, 0)
control_buttons_layout.addWidget(self.unlock_button, 0, 1)
control_buttons_layout.addWidget(self.zero_button, 1, 0)
control_buttons_layout.addWidget(self.reset_button, 1, 1) # Novo botão de reset
control_buttons_layout.addWidget(self.stop_button, 2, 0, 1, 2) # Ocupa duas colunas

axis_layout.addLayout(control_buttons_layout, 6, 0, 1, 2)

# Console de comandos
axis_layout.addWidget(QLabel("Comando GCODE:"), 7, 0, 1, 2)

command_layout = QHBoxLayout()
self.command_input = QLineEdit()

```

```
self.command_input.returnPressed.connect(self.send_command)

self.send_button = QPushButton("Enviar")
self.send_button.clicked.connect(self.send_command)

command_layout.addWidget(self.command_input)
command_layout.addWidget(self.send_button)

axis_layout.addLayout(command_layout, 8, 0, 1, 2)

axis_group.setLayout(axis_layout)
control_status_layout.addWidget(axis_group)

# Painel de status (lado direito)
status_group = QGroupBox("Status da Máquina")
status_layout = QVBoxLayout()

# Widget de exibição de posição
self.position_display = PositionDisplay()
status_layout.addWidget(self.position_display)

# Status da máquina
status_layout.addWidget(QLabel("Estado:"))
self.machine_state_label = QLabel("Desconectado")
self.machine_state_label.setFont(QFont("Arial", 12, QFont.Weight.Bold))
status_layout.addWidget(self.machine_state_label)

# Últimos comandos e respostas
status_layout.addWidget(QLabel("Último Comando:"))
self.last_command_label = QLabel("-")
status_layout.addWidget(self.last_command_label)

status_layout.addWidget(QLabel("Última Resposta:"))
self.last_response_label = QLabel("-")
status_layout.addWidget(self.last_response_label)

# Log de comunicação
status_layout.addWidget(QLabel("Log de Comunicação:"))
self.comm_log = QLineEdit()
self.comm_log.setReadOnly(True)
status_layout.addWidget(self.comm_log)

# Otimizações de comunicação
optimization_group = QGroupBox("Otimizações")
optimization_layout = QVBoxLayout()

self.fast_mode_check = QPushButton("Modo Rápido (Sem Door Check)")
self.fast_mode_check.setCheckable(True)
self.fast_mode_check.clicked.connect(self.toggle_fast_mode)
self.fast_mode_check.setToolTip("Desativa verificação de porta para movimentos mais rápidos")
```

```
optimization_layout.addWidget(self.fast_mode_check)
optimization_group.setLayout(optimization_layout)
status_layout.addWidget(optimization_group)
```

```
status_layout.addStretch()
status_group.setLayout(status_layout)
control_status_layout.addWidget(status_group)
```

```
# Proporção entre painéis de controle e status
control_status_layout.setStretch(0, 3)
control_status_layout.setStretch(1, 2)
```

```
main_layout.addLayout(control_status_layout)
```

```
# Barra de status
self.statusBar = self.statusBar()
self.statusBar.showMessage("Pronto para conexão")
```

```
# Desabilitar controles até que a conexão seja estabelecida
self.toggle_controls(False)
```

```
# Atualizar portas
self.refresh_ports()
```

```
def reset_alarm(self):
```

```
    """Reset o estado de alarme e libera a máquina para movimento"""
    if not self.is_connected or not self.grbl:
        return
```

```
    try:
```

```
        logger.debug("Resetando alarme")
```

```
        # Sequência completa de reset para garantir que o alarme seja cancelado
```

```
        # 1. Envia comando kill alarm
```

```
        self.grbl.send_immediately("$X")
```

```
        # 2. Pausa breve e mais comandos para garantir a recuperação
```

```
        QTimer.singleShot(150, lambda: self.grbl.send_immediately("~")) # Resume
```

```
        # 3. Restaura o modo de movimento atual
```

```
        QTimer.singleShot(300, lambda: self.grbl.send_immediately(self.current_motion_mode))
```

```
        # 4. Soft reset para caso de falha dos comandos acima
```

```
        QTimer.singleShot(400, lambda: self.grbl.send_immediately("\x18")) # Ctrl+X (soft reset)
```

```
        # 5. Restaura novamente o modo após soft reset
```

```
        QTimer.singleShot(600, lambda: self.grbl.send_immediately(self.current_motion_mode))
```

```
        self.statusBar.showMessage("Reset de alarme enviado, máquina deve estar pronta em breve")
```

```
    except Exception as e:
```

```
        logger.error(f"Erro ao resetar alarme: {str(e)}", exc_info=True)
```

```

self.statusBar.showMessage(f"Erro: {str(e)}")

def refresh_ports(self):
    """Detecta automaticamente as portas seriais disponíveis"""
    self.port_combo.clear()

    available_ports = [port.device for port in serial.tools.list_ports.comports()]

    if not available_ports:
        self.statusBar.showMessage("Nenhuma porta serial encontrada")
        return

    self.port_combo.addItems(available_ports)

    # Verificar se COM9 está na lista e selecioná-la
    com9_index = self.port_combo.findText("COM9")
    if com9_index >= 0:
        self.port_combo.setCurrentIndex(com9_index)
        self.statusBar.showMessage("Arduino detectado na porta COM9")
    else:
        self.statusBar.showMessage(f"Portas encontradas: {' '.join(available_ports)}")

def toggle_connection(self):
    """Conecta/desconecta da máquina CNC"""
    if not self.is_connected:
        try:
            port = self.port_combo.currentText()
            if not port:
                self.statusBar.showMessage("Nenhuma porta selecionada")
                return

            # Mensagem de status
            self.statusBar.showMessage(f"Conectando à porta {port}...")

            # CORREÇÃO: Inicializar a comunicação com o GRBL usando a API correta da biblioteca
            def grbl_callback(eventstring, *data):
                """Callback para eventos do GrblStreamer"""
                logger.debug(f"GRBL evento: {eventstring}, dados: {data}")

            # Processar eventos específicos
            if eventstring == "on_stateupdate":
                # Formato: on_stateupdate(state, mpos, wpos)
                if len(data) >= 3:
                    state = data[0]
                    machine_pos = data[1] # (x, y, z)

                    # Atualiza estado da máquina
                    self.machine_status = state

                    # Atualiza posição
                    if len(machine_pos) >= 3:

```

```

        self.current_position = {
            'x': machine_pos[0],
            'y': machine_pos[1],
            'z': machine_pos[2] if len(machine_pos) > 2 else 0
        }
        self.signals.status_update.emit(self.current_position)

    elif eventstring == "on_processed_command":
        # Respostas a comandos
        if len(data) >= 2:
            response = data[1]
            self.signals.response_received.emit(str(response))

    elif eventstring == "on_error":
        # Mensagens de erro
        if data:
            error_msg = str(data[0])
            self.signals.error_message.emit(error_msg)

# Inicializa com o callback
self.grbl = GrblStreamer(grbl_callback)

# Configura logging (opcional)
self.grbl.setup_logging()

# Conecta usando o método correto
self.grbl.cnect(port, 115200)

# Desbloqueia a máquina
self.grbl.send_immediately("$X")

# Iniciar em modo relativo para jog
self.grbl.send_immediately("G91")
self.mode_relative.setChecked(True)
self.mode_absolute.setChecked(False)

# Inicia verificação de status
self.grbl.poll_start()

# Atualizar o estado da interface
self.is_connected = True
self.machine_status = "Idle" # Estado inicial presumido
self.connect_button.setText("Desconectar")
self.machine_state_label.setText("Conectado")
self.statusBar.showMessage(f'Conectado ao GRBL na porta {port}')
self.toggle_controls(True)

except Exception as e:
    logger.error(f'Erro ao conectar: {str(e)}", exc_info=True)
    self.statusBar.showMessage(f'Erro: {str(e)}")
else:

```

```

# Desconectar
self._stop_jog_and_movements()

if self.grbl:
    try:
        # Parar verificação de status
        self.grbl.poll_stop()

        # Desconectar
        self.grbl.disconnect()
        self.grbl = None
    except Exception as e:
        logger.error(f"Erro ao desconectar: {str(e)}", exc_info=True)

self.is_connected = False
self.machine_status = "Desconectado"
self.connect_button.setText("Conectar")
self.machine_state_label.setText("Desconectado")
self.statusBar.showMessage("Desconectado")
self.toggle_controls(False)

def _setup_callbacks(self):
    """Configura callbacks para eventos do GrblStreamer"""
    if not self.grbl:
        return

    # Configurar callbacks para o grbl-streamer
    # (A API exata pode variar com base na implementação da biblioteca)
    self.grbl.register_status_callback(self._on_status_update)
    self.grbl.register_response_callback(self._on_response_received)
    self.grbl.register_error_callback(self._on_error_message)

def _start_status_polling(self):
    """Inicia polling periódico de status"""
    if not self.is_connected or not self.grbl:
        return

    # Criar thread de consulta de status (se a lib não fizer isso automaticamente)
    self.status_thread = threading.Thread(target=self._status_polling_thread, daemon=True)
    self.status_thread.start()

def _status_polling_thread(self):
    """Thread para consulta periódica de status"""
    while self.is_connected and self.grbl:
        try:
            # Requisição de status ao GRBL a cada 200ms
            self.grbl.get_status()
            time.sleep(0.2)
        except Exception as e:
            logger.error(f"Erro na consulta de status: {str(e)}", exc_info=True)
            time.sleep(0.5) # Maior intervalo em caso de erro

```

```

def _on_status_update(self, status_data):
    """Callback quando o status é atualizado pelo grbl-streamer"""
    try:
        # Interpretar os dados de status recebidos do grbl-streamer
        # (O formato exato pode variar com base na implementação da biblioteca)

        # Obtém o estado da máquina
        state = status_data.get('state', 'Unknown')

        # Obtém a posição
        position = status_data.get('position', {})
        if position:
            self.current_position = {
                'x': position.get('x', 0),
                'y': position.get('y', 0),
                'z': position.get('z', 0)
            }

        # Emite sinais para atualizar a interface
        self.machine_status = state
        self.signals.status_update.emit(self.current_position)

    except Exception as e:
        logger.error(f"Erro ao processar status: {str(e)}", exc_info=True)

def _on_response_received(self, response):
    """Callback quando uma resposta é recebida pelo grbl-streamer"""
    try:
        self.last_response = response
        self.signals.response_received.emit(response)
    except Exception as e:
        logger.error(f"Erro ao processar resposta: {str(e)}", exc_info=True)

def _on_error_message(self, error):
    """Callback quando um erro é recebido pelo grbl-streamer"""
    try:
        self.last_error = error
        self.signals.error_message.emit(error)
    except Exception as e:
        logger.error(f"Erro ao processar mensagem de erro: {str(e)}", exc_info=True)

def toggle_controls(self, enabled):
    """Habilita/desabilita os controles com base no estado de conexão"""
    # Controles de eixos e botões de movimento
    self.x_minus_button.setEnabled(enabled)
    self.x_plus_button.setEnabled(enabled)
    self.y_minus_button.setEnabled(enabled)
    self.y_plus_button.setEnabled(enabled)

    # Botões direcionais

```

```
self.dir_up_button.setEnabled(enabled)
self.dir_down_button.setEnabled(enabled)
self.dir_left_button.setEnabled(enabled)
self.dir_right_button.setEnabled(enabled)
self.jog_step.setEnabled(enabled)
```

```
# Campos de entrada
```

```
self.x_distance.setEnabled(enabled)
self.y_distance.setEnabled(enabled)
self.feed_rate.setEnabled(enabled)
self.feed_slider.setEnabled(enabled)
```

```
# Botões de controle
```

```
self.home_button.setEnabled(enabled)
self.unlock_button.setEnabled(enabled)
self.zero_button.setEnabled(enabled)
self.stop_button.setEnabled(enabled)
self.fast_mode_check.setEnabled(enabled)
```

```
# Modos de movimento
```

```
self.mode_absolute.setEnabled(enabled)
self.mode_relative.setEnabled(enabled)
```

```
# Console de comandos
```

```
self.command_input.setEnabled(enabled)
self.send_button.setEnabled(enabled)
```

```
def update_ui_state(self):
```

```
    """Atualiza o estado da interface com base no status da máquina"""
```

```
    # Atualizar texto de status
```

```
    self.machine_state_label.setText(self.machine_status)
```

```
    # Habilitar/desabilitar botão de reset com base no estado
```

```
    is_alarm = self.machine_status == "Alarm"
```

```
    self.reset_button.setEnabled(is_alarm)
```

```
    # Colorir o status da máquina e aplicar estilo ao botão de reset
```

```
    if self.machine_status == "Idle":
```

```
        self.machine_state_label.setStyleSheet("color: green;")
```

```
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
```

```
    elif self.machine_status == "Run" or self.machine_status == "Jog":
```

```
        self.machine_state_label.setStyleSheet("color: blue;")
```

```
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
```

```
    elif self.machine_status == "Alarm":
```

```
        self.machine_state_label.setStyleSheet("color: red;")
```

```
        self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold;")
```

```
    elif self.machine_status == "Hold":
```

```
        self.machine_state_label.setStyleSheet("color: orange;")
```

```
        self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")
```

```
    else:
```

```
        self.machine_state_label.setStyleSheet("")
```

```

self.reset_button.setStyleSheet("background-color: #E0E0E0; color: black;")

# Atualizar display de posição
self.position_display.update_position(self.current_position)

def update_feed_rate(self, value):
    """Atualiza o campo de texto quando o slider muda"""
    self.feed_rate.setText(str(value))

def update_feed_slider(self):
    """Atualiza o slider quando o campo de texto muda"""
    try:
        value = int(self.feed_rate.text())
        # Limitar ao intervalo do slider
        value = max(100, min(5000, value))
        self.feed_slider.setValue(value)
    except ValueError:
        pass

def toggle_fast_mode(self):
    """Ativa/desativa o modo rápido (desabilita verificação de porta)"""
    if not self.is_connected or not self.grbl:
        return

    try:
        if self.fast_mode_check.isChecked():
            # Desativar verificação de porta para movimentos mais rápidos
            self.grbl.send_immediately("$10=0")
            self.statusBar.showMessage("Modo rápido ativado (verificação de porta desativada)")
        else:
            # Reativar verificação de porta
            self.grbl.send_immediately("$10=1")
            self.statusBar.showMessage("Modo rápido desativado (verificação de porta ativada)")
    except Exception as e:
        logger.error(f"Erro ao alternar modo rápido: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro: {str(e)}")

def set_motion_mode(self, mode):
    """Define o modo de movimento (absoluto/passo a passo ou relativo/contínuo)"""
    if not self.is_connected or not self.grbl:
        return

    if mode == "G90":
        self.mode_absolute.setChecked(True)
        self.mode_relative.setChecked(False)
        self.mode_absolute.setText("Passo a Passo (G90)")
        self.mode_relative.setText("Contínuo (G91)")
    else:
        self.mode_absolute.setChecked(False)
        self.mode_relative.setChecked(True)
        self.mode_absolute.setText("Passo a Passo (G90)")

```

```

self.mode_relative.setText("Contínuo (G91)")

# Atualiza o modo atual
self.current_motion_mode = mode

# Envia o comando para a máquina
self.grbl.send_immediately(mode)
self.last_command = mode
self.last_command_label.setText(mode)

# Atualiza a mensagem de status com o novo modo
self.statusBar.showMessage(f"Modo de movimento alterado para {mode}")

def start_jog(self, axis, direction):
    """Inicia movimento jog para um eixo (movimento discreto)"""
    if not self.is_connected or not self.grbl:
        return

    try:
        logger.debug(f"Iniciando jog discreto: eixo={axis}, direção={direction}")

        # Determina a distância de movimento
        if axis == "X":
            distance = float(self.x_distance.text()) * direction
            command = f"G91\nG0 X{distance} F{self.feed_rate.text()}"
        else: # Y
            distance = float(self.y_distance.text()) * direction
            command = f"G91\nG0 Y{distance} F{self.feed_rate.text()}"

        # Envia o comando
        self.grbl.send_gcode(command)
        self.last_command = command
        self.last_command_label.setText(command)

    except Exception as e:
        logger.error(f"Erro ao iniciar jog: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro ao iniciar movimento: {str(e)}")

def stop_jog(self):
    """Para o movimento jog quando o botão é liberado"""
    # Para movimento discreto, não precisamos fazer nada especial
    pass

def _continue_jog_movement(self, axis, direction, distance, feed_rate):
    """Continua o movimento jog enviando comandos incrementais pequenos"""
    if not self.is_connected or not self.grbl or not self.jogging:
        if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
            self.jog_timer.stop()
        return

    # Cria o comando para continuar o movimento

```

```

if axis.upper() == 'X':
    jog_command = f"G00 X{distance} F{feed_rate}"
else: # Y
    jog_command = f"G00 Y{distance} F{feed_rate}"

# Envia o comando usando o método disponível
try:
    self.grbl.send_immediately(jog_command)
except Exception as e:
    logger.error(f"Erro ao continuar jog: {str(e)}", exc_info=True)
    if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
        self.jog_timer.stop()

def _continue_jog(self):
    """Envia comandos contínuos de movimento enquanto o botão está pressionado"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Cria um novo comando usando G1 (movimento linear) em vez de G0
        # G1 é mais suave para movimentos contínuos
        if self.jog_axis.upper() == 'X':
            command = f"G1 X{self.jog_direction * self.jog_step_size} F{self.jog_feed_rate}"
        else: # Y
            command = f"G1 Y{self.jog_direction * self.jog_step_size} F{self.jog_feed_rate}"

        # Envia o comando sem interromper fluxos anteriores
        self.grbl.send_immediately(command)

    except Exception as e:
        logger.error(f"Erro ao continuar jog: {str(e)}", exc_info=True)
        self.jog_timer.stop()

def start_continuous_jog(self, axis, direction):
    """
    Inicia movimento passo a passo (G90) ou contínuo (G91), dependendo do modo atual.
    """
    if not self.is_connected or not self.grbl:
        return

    # Para qualquer jog anterior antes de iniciar o novo
    self.stop_continuous_jog()
    time.sleep(0.05) # Pequena pausa para garantir que o GRBL processe o comando de parada

    try:
        # Obtém o modo atual (G90 = Absoluto, G91 = Relativo)
        is_absolute_mode = (self.current_motion_mode == "G90")

        logger.debug(f"Iniciando jog: eixo={axis}, direção={direction}, modo={'G90 (Absoluto/Passo a Passo)' if is_absolute_mode else 'G91 (Relativo)'}")

        # Obtém a velocidade e o tamanho do passo

```

```

try:
    feed_rate = int(self.feed_rate.text())
    feed_rate = max(100, min(feed_rate, 5000)) # Limita a velocidade para segurança
except ValueError:
    feed_rate = 500 # Valor seguro padrão

try:
    step_size = float(self.jog_step.text())
    step_size = max(0.1, min(step_size, 10)) # Limita o tamanho do passo entre 0.1 e 10mm
except ValueError:
    step_size = 1.0 # Valor padrão seguro

# Movimento com base no modo atual
if is_absolute_mode:
    # G90 MODO ABSOLUTO/PASSO A PASSO: Movimento incremental relativo à posição atual
    # CORREÇÃO: Primeiro garantimos que estamos em modo relativo para o movimento
    self.grbl.send_immediately("G91")

    # Calcula a distância de movimento baseada no passo configurado
    distance = step_size * direction

    # Cria um comando de movimento incremental apesar de estar em modo G90 na interface
    if axis.upper() == 'X':
        jog_command = f"G0 X{distance} F{feed_rate}"
    else: # Y
        jog_command = f"G0 Y{distance} F{feed_rate}"

    # Envia o comando de movimento incremental
    self.grbl.send_immediately(jog_command)
    self.last_command = jog_command
    self.last_command_label.setText(jog_command)

    # Restaura o modo absoluto ao final do movimento
    QTimer.singleShot(100, lambda: self.grbl.send_immediately("G90"))

    # Não define estado de jogging, pois é uma operação única
    self.jogging = False

else:
    # G91 MODO RELATIVO/CONTÍNUO: Movimento contínuo enquanto o botão é pressionado
    # Usar o comando $J= para jog contínuo
    distance_continuous = 50 * direction # Distância grande o suficiente para movimento contínuo

    if axis.upper() == 'X':
        jog_command = f"$J=G91 X{distance_continuous} F{feed_rate}"
    else: # Y
        jog_command = f"$J=G91 Y{distance_continuous} F{feed_rate}"

    # Imediatamente envia o comando de jog
    self.grbl.send_immediately(jog_command)
    self.last_command = jog_command

```

```

self.last_command_label.setText(jog_command)

# Define flags de controle apenas para modo contínuo
self.jogging = True
self.current_jog_axis = axis
self.current_jog_direction = direction

except Exception as e:
    logger.error(f"Erro ao iniciar jog: {str(e)}", exc_info=True)
    self.statusBar.showMessage(f"Erro ao iniciar movimento: {str(e)}")

def stop_continuous_jog(self):
    """
    Para o movimento jog contínuo quando o botão é liberado.
    No modo G90, esta função não faz nada pois o movimento é único.
    """
    if not self.is_connected or not self.grbl or not self.jogging:
        return

    logger.debug("Parando jog contínuo")

    try:
        # Comando de feed hold para parar imediatamente
        self.grbl.send_immediately("!")

        # Imediatamente após, envia comando de ciclo contínuo para liberar o hold
        QTimer.singleShot(50, lambda: self.grbl.send_immediately("~"))

        # Limpa o estado de jog
        self.jogging = False
        self.current_jog_axis = None
        self.current_jog_direction = None

    except Exception as e:
        logger.error(f"Erro ao parar jog contínuo: {str(e)}", exc_info=True)

def _stop_jog_and_movements(self):
    """Para todos os movimentos ativos"""
    if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
        self.jog_timer.stop()

    if not self.is_connected or not self.grbl:
        return

    try:
        # Envia comando de parada
        self.grbl.send_immediately("!")
    except Exception as e:
        logger.error(f"Erro ao parar movimentos: {str(e)}", exc_info=True)

```

```

def _send_resume_after_stop(self):
    """Envia comando de retomada após parada"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Enviar comando de retomada usando o método correto
        self.grbl.send_realtime("~")

    except Exception as e:
        logger.error(f"Erro ao enviar comando de retomada: {str(e)}", exc_info=True)

def home(self):
    """Envia comando para ir para home"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Usar send_immediately
        self.grbl.send_immediately("$H")
        self.last_command = "$H"
        self.last_command_label.setText("$H (Home)")
    except Exception as e:
        logger.error(f"Erro ao enviar comando home: {str(e)}", exc_info=True)

def unlock(self):
    """Desbloqueia a máquina após um alarme"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Usar send_immediately
        self.grbl.send_immediately("$X")
        self.last_command = "$X"
        self.last_command_label.setText("$X (Unlock)")
    except Exception as e:
        logger.error(f"Erro ao enviar comando de desbloqueio: {str(e)}", exc_info=True)

def set_zero(self):
    """Define a posição atual como zero para todos os eixos"""
    if not self.is_connected or not self.grbl:
        return

    try:
        # Usar send_immediately
        self.grbl.send_immediately("G92 X0 Y0")
        self.last_command = "G92 X0 Y0"
        self.last_command_label.setText("G92 X0 Y0 (Set Zero)")
    except Exception as e:
        logger.error(f"Erro ao definir posição zero: {str(e)}", exc_info=True)

```

```

def emergency_stop(self):
    """Para todos os movimentos imediatamente e coloca a máquina em estado seguro"""
    if not self.is_connected or not self.grbl:
        return

    try:
        logger.debug("Executando parada de emergência")

        # Para qualquer timer ativo
        if hasattr(self, 'jog_timer') and self.jog_timer.isActive():
            self.jog_timer.stop()

        # Sequência robusta de parada
        # 1. Feed hold imediato (!)
        self.grbl.send_immediately("!")

        # 2. Soft reset para garantir parada completa
        QTimer.singleShot(100, lambda: self.grbl.send_immediately("\x18")) # Ctrl+X

        # Reset completo de estado
        self.jogging = False
        self.current_jog_axis = None
        self.current_jog_direction = None

        self.last_command = "! (Parada de Emergência)"
        self.last_command_label.setText("! (Parada de Emergência)")
        self.statusBar.showMessage("PARADA DE EMERGÊNCIA acionada! Use o botão 'Reset Alarme' para continuar.")

        # Destaca o botão de reset para orientar o usuário
        self.reset_button.setStyleSheet("background-color: orange; color: white; font-weight: bold; font-size: 14px; border: 2px solid black;")

    except Exception as e:
        logger.error(f"Erro na parada de emergência: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro: {str(e)}")

def send_command(self):
    """Envia um comando G-code personalizado"""
    if not self.is_connected or not self.grbl:
        return

    command = self.command_input.text()
    if not command:
        return

    try:
        # Usar send_immediately em vez de send_gcode
        self.grbl.send_immediately(command)
        self.last_command = command
        self.last_command_label.setText(command)
        self.command_input.clear()
    except Exception as e:

```

```

        logger.error(f"Erro ao enviar comando: {str(e)}", exc_info=True)
        self.statusBar.showMessage(f"Erro ao enviar comando: {str(e)}")

def on_response_received(self, response):
    """Manipula a resposta recebida"""
    self.last_response_label.setText(response)
    self.comm_log.setText(response)

def on_status_update(self, position):
    """Atualiza a exibição de posição na interface"""
    # Esta função é chamada pelo sinal emitido no _on_status_update
    self.position_display.update_position(position)

def on_error_message(self, error):
    """Exibe mensagem de erro na barra de status"""
    self.statusBar.showMessage(error)

def closeEvent(self, event):
    """Manipula o evento de fechamento da janela"""
    # Para todos os movimentos e desconecta
    if self.is_connected and self.grbl:
        self._stop_jog_and_movements()
        try:
            self.grbl.close()
        except Exception:
            pass

    event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = GRBLController()
    window.show()
    sys.exit(app.exec())

```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoil.venv/Lib/site-packages/grbl_streamer

"""

grbl-streamer - Universal interface module for the grbl CNC firmware
 Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful,

but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import logging
import time
import re
import threading
import atexit
```

```
from queue import Queue
```

```
from .interface import Interface
from .callbackloghandler import CallbackLogHandler
```

```
from gcode_machine import GcodeMachine
```

```
class GrblStreamer:
```

```
    """ A universal Grbl CNC firmware interface module for Python3
    providing a convenient high-level API for scripting or integration
    into parent applications like GUI's.
```

There are a number of streaming applications available for the Grbl CNC controller, but none of them seem to be an universal, re-usable standard Python module. GrblStreamer attempts to fill that gap.

See README for usage examples.

Features:

- * Re-usable across projects
- * Non-blocking
- * Asynchronous (event-based) callbacks for the parent application
- * Two streaming modes: Incremental or fast ("counting characters")
- * Defined shutdown
- * G-Code cleanup
- * G-Code variable expansion
- * Dynamic feed override
- * Buffer stashing
- * Job halt and resume

Callbacks:

After assigning your own callback function (callback = ...) you will receive the following signals:

on_boot

: Emitted whenever Grbl boots (e.g. after a soft reset).

: No arguments.

on_read

: Emitted whenever a line is read by GrblStreamer

: 1 argument: line

on_disconnected

: Emitted whenever the serial port has been closed.

: No arguments

on_log

: Emitted for informal logging or debugging messages.

: 1 argument: LogRecord instance

on_line_sent

: Emitted whenever a line is actually sent to Grbl.

: 2 arguments: job_line_number, line

on_bufsize_change

: Emitted whenever lines have been appended to the buffer

: 1 argument: linecount

on_line_number_change

: Emitted whenever the current buffer position has been changed

: 1 argument: line_number

on_processed_command

: Emitted whenever Grbl confirms a command with "ok" and is now being executed physically

: 2 arguments: processed line number, processed line

on_alarm

: Emitted whenever Grbl sends an "ALARM" line

: 1 argument: the full line Grbl sent

on_error

: Emitted whenever Grbl sends an "ERROR" line

: 3 arguments: the full line Grbl sent, the line that caused the error, the line number in the buffer that caused the error

on_rx_buffer_percent

: Reports Grbl's serial receive buffer fill in percent. Emitted frequently while streaming.

: 1 argument: percentage integer from 0 to 100

on_progress_percent

: Reports the completion of the current job/buffer in percent. Emitted frequently while streaming.

: 1 argument: percentage integer from 0 to 100

on_job_completed

: Emitted when the current job/buffer has been streamed and physically executed entirely

on_stateupdate

: Emitted whenever Grbl's state has changed

: 3 arguments: Grbl's mode ('Idle', 'Run' etc.), machine position tuple, working position tupe

on_hash_stateupdate

: Emitted after Grbl's 'hash' EEPROM settings (`\$#`) have been received

: 1 argument: dict of the settings

on_settings_downloaded

: Emitted after Grbl's EEPROM settings (`\$\$`) have been received

: 1 argument: dict of the settings

on_gcode_parser_stateupdate

: Emitted after Grbl's G-Code parser state has been received

: 1 argument: list of the state variables

on_simulation_finished

: Emitted when GrblStreamer's target is set to "simulator" and the job is executed.

: 1 argument: list of all G-Code commands that would have been sent to Grbl

on_vars_change

: Emitted after G-Code is loaded into the buffer and variables have been detected

: 1 argument: a dict of the detected variables

on_preprocessor_feed_change

: Emitted when a F keyword is parsed from the G-Code.

: 1 argument: the feed rate in mm/min

"""

__version__ = "0.5.0"

def __init__(self, callback, name="mygrbl"):

"""Straightforward initialization tasks.

@param callback

Set your own function that will be called when a number of asynchronous events happen. Useful for UI's. The default function will just log to stdout.

This callback function will receive two arguments. The first is a string giving a label of the event, and the second is a variable argument list `\*args` containing data pertaining to the event.

Note that this function may be called from a Thread.

@param name

An informal name of the instance. Useful if you are running several instances to control several CNC machines at once. It is only used for logging output and UI messages.

"""

```

# @var name
# Set an informal name of the instance. Useful if you are
# running several instances to control several CNC machines at
# once. It is only used for logging output and UI messages.
self.name = name

# @var cmode
# Get Grbl's current mode.
# Will be strings 'Idle', 'Check', 'Run'
self.cmode = None

# @var cmpos
# Get a 3-tuple containing the current coordinates relative
# to the machine origin.
self.cmpo = (0, 0, 0)

# @var cwpos
# Get a 3-tuple containing the current working coordinates.
# Working coordinates are relative to the currently selected
# coordinate system.
self.cwpos = (0, 0, 0)

# @var gps
# Get list of 12 elements containing the 12 Gcode Parser State
# variables of Grbl which are obtained by sending the raw
# command `G`. Will be available after setting
# `hash_state_requested` to True.
self.gps = [
    '0', # motion mode
    '54', # current coordinate system
    '17', # current plane mode
    '21', # units
    '90', # current distance mode
    '94', # feed rate mode
    '0', # program mode
    '0', # spindle state
    '5', # coolant state
    '0', # tool number
    '99', # current feed
    '0', # spindle speed
]

# @var poll_interval
# Set an interval in seconds for polling Grbl's state via
# the `?` command. The Grbl Wiki recommends to set this no lower
# than 0.2 (5 per second).
self.poll_interval = 0.2

# @var settings
# Get a dictionary of Grbl's EEPROM settings which can be read

```

```

# after sending the `$$` command, or more conveniently after
# calling the method `request_settings()` of this class.
self.settings = {
    130: {'val': '1000', 'cmt': 'width'},
    131: {'val': '1000', 'cmt': 'height'}
}

# @var settings_hash
# Get a dictionary of Grbl's 'hash' settings (also stored in the
# EEPROM) which can be read after sending the `$#` command. It
# contains things like coordinate system offsets. See Grbl
# documentation for more info. Will be available shortly after
# setting `hash_state_requested` to `True`.
self.settings_hash = {
    'G54': (0, 0, 0),
    'G55': (0, 0, 0),
    'G56': (0, 0, 0),
    'G57': (0, 0, 0),
    'G58': (0, 0, 0),
    'G59': (0, 0, 0),
    'G28': (0, 0, 0),
    'G30': (0, 0, 0),
    'G92': (0, 0, 0),
    'TLO': 0,
    'PRB': (0, 0, 0),
}

# @var gcode_parser_state_requested
# Set this variable to `True` to receive a callback with the
# event string "on_gcode_parser_stateupdate" containing
# data that Grbl sends after receiving the `$G` command.
# After the callback, this variable reverts to `False`.
self.gcode_parser_state_requested = False

# @var hash_state_requested
# Set this variable to `True` to receive a callback with the
# event string "on_hash_stateupdate" containing
# the requested data. After the callback, this variable reverts
# to `False`.
self.hash_state_requested = False

# @var logger
# The logger used by this class. The default is Python's own
# logger module. Use `setup_logging()` to attach custom
# log handlers.
self.logger = logging.getLogger('GrblStreamer')
self.logger.setLevel(logging.DEBUG)
self.logger.propagate = False

# @var target
# Set this to change the output target. Default is "firmware"

```

```

# which means the serial port. Another target is "simulator",
# you will receive a callback with even string
# "on_simulation_finished" and a buffer of the G-Code commands
# that would have been sent out to Grbl.
# TODO: Add "file" target.
self.target = 'firmware'

# @var connected
# `True` when connected to Grbl (after boot), otherwise `False`
self.connected = False

# @var preprocessor
# All G-code commands will go through the preprocessor
# before they are sent out via the serial port. The preprocessor
# keeps track of, and can dynamically change, feed rates, as well
# as substitute variables. It has its own state and callback
# functions.
self.preprocessor = GcodeMachine()
self.preprocessor.callback = self._preprocessor_callback

# @var travel_dist_buffer
# The total distance of all G-Codes in the buffer.
self.travel_dist_buffer = {}

# @var travel_dist_current
# The currently travelled distance. Can be used to calculate ETA.
self.travel_dist_current = {}

# @var is_standstill
# If the machine is currently not moving
self.is_standstill = False

self._ifacepath = None
self._last_setting_number = 132

self._last_cmode = None
self._last_cmpos = (0, 0, 0)
self._last_cwpos = (0, 0, 0)

self._standstill_watchdog_increment = 0

self._rx_buffer_size = 128
self._rx_buffer_fill = []
self._rx_buffer_backlog = []
self._rx_buffer_backlog_line_number = []
self._rx_buffer_fill_percent = 0

self._current_line = ""
self._current_line_sent = True
self._streaming_mode = None
self._wait_empty_buffer = False

```

```
self.streaming_complete = True
self.job_finished = True
self._streaming_src_end_reached = True
self._streaming_enabled = True
self._error = False
self._incremental_streaming = False
```

```
self.buffer = []
self.buffer_size = 0
self._current_line_nr = 0
```

```
self.buffer_stash = []
self.buffer_size_stash = 0
self._current_line_nr_stash = 0
```

```
self._poll_keep_alive = False
self._iface_read_do = False
```

```
self._thread_polling = None
self._thread_read_iface = None
```

```
self._iface = None
self._queue = Queue()
```

```
self._loghandler = None
```

```
# general-purpose counter for timing tasks inside of _poll_state
self._counter = 0
```

```
self._callback = callback
```

```
atexit.register(self.disconnect)
```

```
# supply defaults to GUI to make it operational
self._callback('on_settings_downloaded', self.settings)
self._callback('on_hash_stateupdate', self.settings_hash)
self.preprocessor.cs_offsets = self.settings_hash
self._callback('on_gcode_parser_stateupdate', self.gps)
```

```
def setup_logging(self, handler=None):
    """Assign a custom log handler.
```

GrblStreamer can be used in both console applications as well as integrated in other projects like GUI's. Therefore, logging to stdout is not always useful. You can pass a custom log message handler to this method. If no handler is passed in, the default handler is an instance of class `CallbackLogHandler` (see file `callback\_loghandler.py` included in this module). CallbackLogHandler will deliver logged strings as callbacks to the parent application, the event string will be "on_log".

@param handler=None

An instance of a subclass inheriting from `logging.StreamHandler`

"""

The default log handler shipped with this module will call

self._callback() with first parameter "on_log" and second

parameter with the logged string.

if handler is not None:

self._loghandler = handler

else:

lh = CallbackLogHandler()

lh.callback = self._callback

self._loghandler = lh

self.logger.addHandler(self._loghandler)

def cnect(self, path=None, baudrate=115200):

"""

Connect to the RS232 port of the Grbl controller.

@param path=None Path to the device node

This is done by instantiating a RS232 class, included in this

module, which by itself block-listens (in a thread) to

asynchronous data sent by the Grbl controller.

"""

if path is None or path.strip() == "":

return

else:

self._ifacepath = path

if self._iface is None:

self.logger.debug('{}: Setting up interface on {}'.format(self.name, self._ifacepath))

self._iface = Interface('iface_' + self.name, self._ifacepath, baudrate)

self._iface.start(self._queue)

else:

self.logger.info('{}: Cannot start another interface. There is already an interface {}'.format(self.name, self._iface))

self._iface_read_do = True

self._thread_read_iface = threading.Thread(target=self._onread)

self._thread_read_iface.start()

self.softreset()

def disconnect(self):

"""

This method provides a controlled shutdown and cleanup of this module.

It stops all threads, joins them, then closes the serial

connection. For a safe shutdown of Grbl you may also want to

call `softreset()` before you call this method.

"""

```
if not self.is_connected():
    return
```

```
self.poll_stop()
```

```
self._iface.stop()
self._iface = None
```

```
self.logger.debug('{}: Please wait until reading thread has joined...'.format(self.name))
```

```
self._iface_read_do = False
```

```
# The thread will not join without putting a last queue message:
```

```
self._queue.put("dummy_msg_for_joining_thread")
```

```
self._thread_read_iface.join()
```

```
self.logger.debug('{}: Reading thread successfully joined.'.format(self.name))
```

```
self.connected = False
```

```
self._callback('on_disconnected')
```

```
def softreset(self):
```

"""

Immediately sends `Ctrl-X` to Grbl.

"""

```
self._iface.write('\x18') # Ctrl-X
```

```
self.update_preprocessor_position()
```

```
def abort(self):
```

"""

An alias for `softreset()`.

"""

```
if not self.is_connected():
```

```
    return
```

```
self.softreset()
```

```
def hold(self):
```

"""

Immediately sends the feed hold command (exclamation mark) to Grbl.

"""

```
if not self.is_connected():
```

```
    return
```

```
self._iface_write('!')
```

```
def resume(self):
```

"""

Immediately send the resume command (tilde) to Grbl.

"""

```
if not self.is_connected():
```

```
    return
```

```
self._iface_write('~')
```

```
def killalarm(self):
```

```
    """
```

```
    Immediately send the kill alarm command ($X) to Grbl.
```

```
    """
```

```
    self._iface_write('$X\n')
```

```
def homing(self):
```

```
    """
```

```
    Immediately send the homing command ($H) to Grbl.
```

```
    """
```

```
    self._iface_write('$H\n')
```

```
def poll_start(self):
```

```
    """
```

```
    Starts forever polling Grbl's status with the `?` command. The  
    polling interval is controlled by setting `self.poll_interval`.
```

```
    You will receive callbacks with the "on_stateupdate" event  
    string containing 3 data parameters self.cmode, self.cmpos,  
    self.cwpos, but only when Grbl's state CHANGES.
```

```
    """
```

```
    if not self.is_connected():
```

```
        return
```

```
    self._poll_keep_alive = True
```

```
    self._last_cmode = None
```

```
    if self._thread_polling is None:
```

```
        self._thread_polling = threading.Thread(target=self._poll_state)
```

```
        self._thread_polling.start()
```

```
        self.logger.debug('{}: Polling thread started'.format(self.name))
```

```
    else:
```

```
        self.logger.debug('{}: Polling thread already running...'.format(self.name))
```

```
def poll_stop(self):
```

```
    """
```

```
    Stops polling that has been started with `poll_start()`
```

```
    """
```

```
    if not self.is_connected():
```

```
        return
```

```
    if self._thread_polling is not None:
```

```
        self._poll_keep_alive = False
```

```
        self.logger.debug('{}: Please wait until polling thread has joined...'.format(self.name))
```

```
        self._thread_polling.join()
```

```
        self.logger.debug('{}: Polling thread has successfully joined...'.format(self.name))
```

```
    else:
```

```
        self.logger.debug('{}: Cannot stop polling thread because none running.'.format(self.name))
```

```
    self._thread_polling = None
```

```
def set_feed_override(self, val):
```

```
    """
```

Enable or disable the feed override feature.

@param val

Pass `True` or `False` as argument to enable or disable dynamic feed override. After passing `True`, you may set the requested feed by calling `self.request\_feed()` one or many times.

```
    """
```

```
    self.preprocessor.do_feed_override = val
```

```
def request_feed(self, requested_feed):
```

```
    """
```

Override the feed speed.

Effective only when you set `set\_feed\_override(True)`.

@param requested_feed

The feed speed in mm/min.

```
    """
```

```
    self.preprocessor.request_feed = float(requested_feed)
```

@property

```
def incremental_streaming(self):
```

```
    return self._incremental_streaming
```

@incremental_streaming.setter

```
def incremental_streaming(self, value):
```

```
    """
```

Incremental streaming means that a new command is sent to Grbl only after Grbl has responded with 'ok' to the last sent command. This is necessary to flash \$ settings to the EEPROM.

Non-incremental streaming means that Grbl's 100-some-byte receive buffer will be kept as full as possible at all times, to give its motion planner system enough data to work with. This results in smoother and faster axis motion. This is also called 'advanced streaming protocol based on counting characters' -- see Grbl Wiki.

You can dynamically change the streaming method even during streaming, while running a job. The buffer fill percentage will reflect the change even during streaming.

@param onoff

Set to `True` to use incremental streaming. Set to `False` to use non-incremental streaming. The default on module startup is `False`.

```
    """
```

```
    self._incremental_streaming = value
```

```
    if self._incremental_streaming:
```

```
self._wait_empty_buffer = True
self.logger.debug('{}: Incremental streaming set to {}'.format(self.name, self._incremental_streaming))
```

```
def send_immediately(self, line):
```

```
    """
```

G-Code command strings passed to this function will bypass buffer management and will be sent to Grbl immediately.

Use this function with caution: Only send when you are sure Grbl's receive buffer can handle the data volume and when it doesn't interfere with currently running streams. Only send single commands at a time.

Applications of this method: manual jogging, coordinate settings etc.

@param line

A string of a single G-Code command to be sent. Doesn't have to be \n terminated.

```
    """
```

```
    bytes_in_firmware_buffer = sum(self._rx_buffer_fill)
```

```
    if bytes_in_firmware_buffer > 0:
```

```
        self.logger.error('Firmware buffer has {:d} unprocessed bytes in it. Will not send {}'.format(bytes_in_firmware_buffer,
                                                                                                     return
```

```
    if self.cmode == 'Alarm':
```

```
        self.logger.error('Grbl is in ALARM state. Will not send {}'.format(line))
        return
```

```
    if self.cmode == 'Hold':
```

```
        self.logger.error('Grbl is in HOLD state. Will not send {}'.format(line))
        return
```

```
    if "$#" in line:
```

```
        # The PRB response is sent for $# as well as when probing.
        # Regular querying of the hash state needs to be done like this,
        # otherwise the PRB response would be interpreted as a probe answer
        self.hash_state_requested = True
        return
```

```
    self.preprocessor.set_line(line)
```

```
    self.preprocessor.strip()
```

```
    self.preprocessor.tidy()
```

```
    self.preprocessor.parse_state()
```

```
    self.preprocessor.override_feed()
```

```
    self._iface_write(self.preprocessor.line + '\n')
```

```
def write(self, lines):
```

```
    """
```

G-Code command strings passed to this function will be appended to the current queue buffer, however a job is not started

automatically. You have to call `job\_run()` to start streaming.

You can call this method repeatedly, e.g. for submitting chunks of G-Code, even while a job is running.

@param lines

A list or newline-separated string of G-Code commands.

"""

```
self._load_lines_into_buffer(lines)
```

```
def load_file(self, filename):
```

"""

Pass a filename to this function to load its contents into the buffer.

A job is not started automatically. Call `job\_run` to start the job.

@param filename

A string giving the relative or absolute file path

"""

```
with open(filename) as f:
```

```
    self._load_lines_into_buffer(f.read())
```

```
def job_run(self, linenr=None):
```

"""

Run the current job, i.e. start streaming the current buffer from a specific line number.

@param linenr

If `linenr` is not specified, start streaming from the current buffer position (`self.current\_line\_number`).

If `linenr` is specified, start streaming from this line.

"""

```
if self.buffer_size == 0:
```

```
    self.logger.warning('{}: Cannot run job. Nothing in the buffer!'.format(self.name))
```

```
    return
```

```
if linenr:
```

```
    self.current_line_number = linenr
```

```
self.travel_dist_current = {}
```

```
self._set_streaming_src_end_reached(False)
```

```
self._set_streaming_complete(False)
```

```
self._streaming_enabled = True
```

```
self._current_line_sent = True
```

```
self._set_job_finished(False)
```

```
self._stream()
```

```
def job_halt(self):
```

"""

Stop streaming. Grbl still will continue processing

all G-Code in its internal serial receive buffer.

"""

self._streaming_enabled = False

def job_new(self):

"""

Start a new job. A "job" in our terminology means the buffer's contents. This function will empty the buffer, set the buffer position to 0, and reset internal state.

"""

del self.buffer[:]

self.buffer_size = 0

self._current_line_nr = 0

self._callback('on_line_number_change', 0)

self._callback('on_bufsize_change', 0)

self._set_streaming_complete(True)

self.job_finished = True

self._set_streaming_src_end_reached(True)

self._error = False

self._current_line = "

self._current_line_sent = True

self.travel_dist_buffer = {}

self.travel_dist_current = {}

self._callback('on_vars_change', self.preprocessor.vars)

@property

def current_line_number(self):

return self._current_line_nr

@current_line_number.setter

def current_line_number(self, linenr):

if linenr < self.buffer_size:

self._current_line_nr = linenr

self._callback('on_line_number_change', self._current_line_nr)

def request_settings(self):

"""

This will send `\$\$` to Grbl and you will receive a callback with the argument 1 "on_settings_downloaded", and argument 2 a dict of the settings.

"""

self._iface_write('\$\$\n')

def do_buffer_stash(self):

"""

Stash the current buffer and position away and initialize a new job. This is useful if you want to stop the current job, stream changed \$ settings to Grbl, and then resume the job where you left off. See also `self.buffer\_unstash()`.

"""

```
self.buffer_stash = list(self.buffer)
self.buffer_size_stash = self.buffer_size
self._current_line_nr_stash = self._current_line_nr
self.job_new()
```

```
def do_buffer_unstash(self):
```

```
    """
```

```
    Restores the previous stashed buffer and position.
```

```
    """
```

```
self.buffer = list(self.buffer_stash)
self.buffer_size = self.buffer_size_stash
self.current_line_number = self._current_line_nr_stash
self._callback('on_bufsize_change', self.buffer_size)
```

```
def update_preprocessor_position(self):
```

```
    # keep preprocessor informed about current working pos
```

```
self.preprocessor.position_m = list(self.cmpos)
```

```
    # self.preprocessor.target = list(self.cmpos)
```

```
def _preprocessor_callback(self, event, *data):
```

```
    if event == 'on_preprocessor_var_undefined':
```

```
        self.logger.critical(
```

```
            'HALTED JOB BECAUSE UNDEFINED VAR {}'.format(data[0])
```

```
        )
```

```
        self._set_streaming_src_end_reached(True)
```

```
        self.job_halt()
```

```
    else:
```

```
        self._callback(event, *data)
```

```
def _stream(self):
```

```
    if self._streaming_src_end_reached:
```

```
        return
```

```
    if not self._streaming_enabled:
```

```
        return
```

```
    if self.target == 'firmware':
```

```
        if self._incremental_streaming:
```

```
            self._set_next_line()
```

```
            if not self._streaming_src_end_reached:
```

```
                self._send_current_line()
```

```
            else:
```

```
                self._set_job_finished(True)
```

```
        else:
```

```
            self._fill_rx_buffer_until_full()
```

```
    elif self.target == 'simulator':
```

```
        buf = []
```

```
        while not self._streaming_src_end_reached:
```

```
            self._set_next_line(True)
```

```
            if self._current_line_nr < self.buffer_size:
```

```

        buf.append(self._current_line)

    # one line still to go
    self._set_next_line(True)
    buf.append(self._current_line)

    self._set_job_finished(True)
    self._callback('on_simulation_finished', buf)

def _fill_rx_buffer_until_full(self):
    while True:
        if self._current_line_sent:
            self._set_next_line()

        if not self._streaming_src_end_reached and self._rx_buf_can_receive_current_line():
            self._send_current_line()
        else:
            break

def _set_next_line(self, send_comments=False):
    progress_percent = int(100 * self._current_line_nr / self.buffer_size)
    self._callback('on_progress_percent', progress_percent)

    if self._current_line_nr < self.buffer_size:
        # still something in _buffer, pop it
        line = self.buffer[self._current_line_nr].strip()
        self.preprocessor.set_line(line)
        self.preprocessor.substitute_vars()
        self.preprocessor.parse_state()
        self.preprocessor.override_feed()
        self.preprocessor.scale_spindle()

        if send_comments:
            self._current_line = self.preprocessor.line + self.preprocessor.comment
        else:
            self._current_line = self.preprocessor.line

        self._current_line_sent = False
        self._current_line_nr += 1

        self.preprocessor.done()

    else:
        # the buffer is empty, nothing more to read
        self._set_streaming_src_end_reached(True)

def _send_current_line(self):
    if self._error:
        self.logger.error('Firmware reported error. Halting.')
        self._set_streaming_src_end_reached(True)
        self._set_streaming_complete(True)

```

```
return
```

```
self._set_streaming_complete(False)
```

```
# +1 for \n which we will append below
line_length = len(self._current_line) + 1
self._rx_buffer_fill.append(line_length)
self._rx_buffer_backlog.append(self._current_line)
self._rx_buffer_backlog_line_number.append(self._current_line_nr)
self._iface_write(self._current_line + '\n')
```

```
self._current_line_sent = True
self._callback('on_line_sent', self._current_line_nr,
               self._current_line)
```

```
def _rx_buf_can_receive_current_line(self):
    rx_free_bytes = self._rx_buffer_size - sum(self._rx_buffer_fill)
    required_bytes = len(self._current_line) + 1 # +1 because \n
    return rx_free_bytes >= required_bytes
```

```
def _rx_buffer_fill_pop(self):
    if len(self._rx_buffer_fill) > 0:
        self._rx_buffer_fill.pop(0)
        processed_command = self._rx_buffer_backlog.pop(0)
        ln = self._rx_buffer_backlog_line_number.pop(0) - 1
        self._callback('on_processed_command', ln, processed_command)
```

```
if self._streaming_src_end_reached and len(self._rx_buffer_fill) == 0:
    self._set_job_finished(True)
    self._set_streaming_complete(True)
```

```
def _iface_write(self, line):
    self._callback('on_write', line)
    if self._iface:
        num_written = self._iface.write(line)
```

```
def _onread(self):
    while self._iface_read_do:
        line = self._queue.get()

        if len(line) > 0:
            if line[0] == '<':
                self._update_state(line)

            elif line == 'ok':
                self._handle_ok()

            elif re.match(r'^\[G[0123] .*', line):
                self._update_gcode_parser_state(line)
                self._callback("on_read", line)
```

```

elif line.startswith('[MSG:'):
    # nothing to do here
    pass

elif re.match(r'^[...:.*', line):
    self._callback('on_read', line)
    self._update_hash_state(line)

    if 'PRB' in line:
        # last line
        self._callback('on_hash_stateupdate', self.settings_hash)
        self.preprocessor.cs_offsets = self.settings_hash
        self._callback('on_probe', self.settings_hash['PRB'])

elif 'ALARM' in line:
    # grbl for some reason doesn't respond to ? polling
    # when there is an alarm due to soft limits
    self.cmode = 'Alarm'
    self._callback('on_stateupdate', self.cmode, self.cmpos, self.cwpos)
    self._callback('on_read', line)
    self._callback('on_alarm', line)

elif 'error' in line:
    # self.logger.debug("ERROR")
    self._error = True
    # self.logger.debug("%s: _rx_buffer_backlog at time of error: %s", self.name, self._rx_buffer_backlog)
    if len(self._rx_buffer_backlog) > 0:
        problem_command = self._rx_buffer_backlog[0]
        problem_line = self._rx_buffer_backlog_line_number[0]
    else:
        problem_command = 'unknown'
        problem_line = -1
    self._callback('on_error', line, problem_command, problem_line)
    self._set_streaming_complete(True)
    self._set_streaming_src_end_reached(True)

elif "Grbl " in line:
    self._callback('on_read', line)
    self._on_bootup()
    self.hash_state_requested = True
    self.request_settings()
    self.gcode_parser_state_requested = True

else:
    m = re.match(r'\$(.*)=(.*) \((.*)\)', line)
    if m:
        key = int(m.group(1))
        val = m.group(2)
        comment = m.group(3)
        self.settings[key] = {
            'val': val,

```

```

        'cmt': comment
    }
    self._callback('on_read', line)
    if key == self._last_setting_number:
        self._callback('on_settings_downloaded', self.settings)
    else:
        self._callback('on_read', line)
        # self.logger.info("{}: Could not parse settings: {}".format(self.name, line))

```

```

def _handle_ok(self):
    if not self.streaming_complete:
        self._rx_buffer_fill_pop()
        if not (self._wait_empty_buffer and len(self._rx_buffer_fill) > 0):
            self._wait_empty_buffer = False
        self._stream()

```

```

self._rx_buffer_fill_percent = int(100 - 100 * (self._rx_buffer_size - sum(self._rx_buffer_fill)) / self._rx_buffer_size)
self._callback('on_rx_buffer_percent', self._rx_buffer_fill_percent)

```

```

def _on_bootup(self):
    self._onboot_init()
    self.connected = True
    self.logger.debug('{}: Grbl has booted!'.format(self.name))
    self._callback('on_boot')

```

```

def _update_hash_state(self, line):
    line = line.replace(']', ' ').replace('[', ' ')
    parts = line.split(':')
    key = parts[0]
    tpl_str = parts[1].split(',')
    tpl = tuple([float(x) for x in tpl_str])
    self.settings_hash[key] = tpl

```

```

def _update_gcode_parser_state(self, line):
    m = re.match(r'[G(\d) G(\d\d) G(\d\d) G(\d\d) G(\d\d) G(\d\d) M(\d) M(\d) M(\d) T(\d) F([\d.-]*?) S([\d.-]*?)\]', line)
    if m:
        self.gps[0] = m.group(1) # motionmode
        self.gps[1] = m.group(2) # current coordinate system
        self.gps[2] = m.group(3) # plane
        self.gps[3] = m.group(4) # units
        self.gps[4] = m.group(5) # dist
        self.gps[5] = m.group(6) # feed rate mode
        self.gps[6] = m.group(7) # program mode
        self.gps[7] = m.group(8) # spindle state
        self.gps[8] = m.group(9) # coolant state
        self.gps[9] = m.group(10) # tool number
        self.gps[10] = m.group(11) # current feed
        self.gps[11] = m.group(12) # current rpm
        self._callback('on_gcode_parser_stateupdate', self.gps)

    self.update_preprocessor_position()

```

else:

```
self.logger.error('{}: Could not parse gcode parser report: "{}".format(self.name, line))
```

def _update_state(self, line):

```
m = re.match(r'<(.*?),MPos:(.*?),WPos:(.*?)>', line)
```

if m is not None:

```
# GRBL v0.9
```

```
# <Idle,MPos:0.000,3.000,0.000,WPos:0.000,3.000,0.000>
```

```
self.cmode = m.group(1)
```

```
mpos_parts = m.group(2).split(',')
```

```
wpos_parts = m.group(3).split(',')
```

```
self.cmpo = (float(mpos_parts[0]), float(mpos_parts[1]), float(mpos_parts[2]))
```

```
self.cwpos = (float(wpos_parts[0]), float(wpos_parts[1]), float(wpos_parts[2]))
```

else:

```
# GRBL v1.1
```

```
# <Idle|MPos:0.0000,0.0000,0.0000|Bf:15,128|FS:0.0,0|WCO:0.0000,0.0000,0.0000>
```

```
m = re.match(r'<(.*?)\\MPos:(.*?)\\|', line)
```

if m is not None:

```
# machine position reported (when $10=1)
```

```
self.cmode = m.group(1)
```

```
mpos_parts = m.group(2).split(',')
```

```
self.cmpo = (float(mpos_parts[0]), float(mpos_parts[1]), float(mpos_parts[2]))
```

else:

```
m = re.match(r'<(.*?)\\WPos:(.*?)\\|', line)
```

if m is not None:

```
# work position reported (when $10=0)
```

```
self.cmode = m.group(1)
```

```
wpos_parts = m.group(2).split(',')
```

```
self.cwpos = (float(wpos_parts[0]), float(wpos_parts[1]), float(wpos_parts[2]))
```

else:

```
self.logger.error('{}: Could not parse MPos or WPos: "{}".format(self.name, line))
```

```
return
```

```
# if we made it here, we parsed MPos or WPos or both
```

if (

```
self.cmode != self._last_cmode or
```

```
self.cmpo != self._last_cmpo or
```

```
self.cwpos != self._last_cwpos
```

):

```
self._callback(
```

```
    'on_stateupdate',
```

```
    self.cmode,
```

```
    self.cmpo,
```

```
    self.cwpos)
```

```
if self.streaming_complete and self.cmode == 'Idle':
```

```
    self.update_preprocessor_position()
```

```
    self.gcode_parser_state_requested = True
```

```
if (self.cmpo != self._last_cmpo):
```

```
    if self.is_standstill:
```

```
        self._standstill_watchdog_increment = 0
```

```

        self.is_standstill = False
        self._callback('on_movement')
    else:
        # no change in positions
        self._standstill_watchdog_increment += 1

    if not self.is_standstill and self._standstill_watchdog_increment > 10:
        # machine is not moving
        self.is_standstill = True
        self._callback('on_standstill')

    self._last_cmode = self.cmode
    self._last_cmpos = self.cmpos
    self._last_cwpos = self.cwpos

def _load_line_into_buffer(self, line):
    self.preprocessor.set_line(line)
    split_lines = self.preprocessor.split_lines()

    for l1 in split_lines:
        self.preprocessor.set_line(l1)
        self.preprocessor.strip()
        self.preprocessor.tidy()
        self.preprocessor.parse_state()
        self.preprocessor.find_vars()
        fractionized_lines = self.preprocessor.fractionize()

        for l2 in fractionized_lines:
            self.buffer.append(l2)
            self.buffer_size += 1

    self.preprocessor.done()

def _load_lines_into_buffer(self, lines):
    if type(lines) is str:
        lines = lines.split('\n')

    for line in lines:
        self._load_line_into_buffer(line)

    self._callback('on_bufsize_change', self.buffer_size)
    self._callback('on_vars_change', self.preprocessor.vars)

def is_connected(self):
    if not self.connected:
        # self.logger.info("{}: Not yet connected".format(self.name))
        pass
    return self.connected

def _onboot_init(self):
    # called after boot. Mimics Grbl's initial state after boot.

```

```

del self._rx_buffer_fill[:]
del self._rx_buffer_backlog[:]
del self._rx_buffer_backlog_line_number[:]
self._set_streaming_complete(True)
self._set_job_finished(True)
self._set_streaming_src_end_reached(True)
self._error = False
self._current_line = ""
self._current_line_sent = True
self._clear_queue()
self.is_standstill = False
self.preprocessor.reset()
self._callback('on_progress_percent', 0)
self._callback('on_rx_buffer_percent', 0)

```

```

def _clear_queue(self):
    try:
        junk = self._queue.get_nowait()
        self.logger.debug('Discarding junk %s', junk)
    except:
        # self.logger.debug("Queue was empty")
        pass

```

```

def _poll_state(self):
    while self._poll_keep_alive:
        self._counter += 1

        if self.hash_state_requested:
            self.get_hash_state()
            self.hash_state_requested = False

        elif self.gcode_parser_state_requested:
            self.get_gcode_parser_state()
            self.gcode_parser_state_requested = False

        else:
            self._get_state()

        time.sleep(self.poll_interval)

    self.logger.debug('{}: Polling has been stopped'.format(self.name))

```

```

def _get_state(self):
    self._iface.write('?')

```

```

def get_gcode_parser_state(self):
    self._iface_write('$G\n')

```

```

def get_hash_state(self):
    if self.cmode == 'Hold':
        self.logger.info('{}: $# command not supported in Hold mode.'.format(self.name))

```

```

        return

    self._iface_write('$#\n')

def _set_streaming_src_end_reached(self, a):
    self._streaming_src_end_reached = a

def _set_streaming_complete(self, a):
    self.streaming_complete = a

def _set_job_finished(self, a):
    self.job_finished = a
    if a:
        self._callback('on_job_completed')

def _default_callback(self, status, *args):
    print('DEFAULT CALLBACK', status, args)

```

File: callbackloghandler.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoil.venv/Lib/site-packages/grbl_streamer

"""

grbl-streamer - Universal interface module for the grbl CNC firmware
 Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import logging
```

```

class CallbackLogHandler(logging.StreamHandler):
    def __init__(self, cb=None):
        super(CallbackLogHandler, self).__init__()
        self.callback = cb

    def emit(self, record):

```

```

if self.callback:
    #txt = "{0} {1}".format(record.level, record.msg)
    #print(self.format(record))
    #self.callback("on_log", self.format(record))
    #self.callback("on_log", record.levelno, record.msg % record.args)
    self.callback('on_log', record)
else:
    logging.StreamHandler.emit(self, record)

```

File: interface.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoil/.venv/Lib/site-packages/grbl_streamer

"""

grbl-streamer - Universal interface module for the grbl CNC firmware
Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with pyglpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```

import serial
import threading
import logging

```

```

class Interface:
    """Implements opening, closing, writing and threaded reading
    from the serial port. Read data are put into a Thread Queue.
    """

```

```

def __init__(self, name, path, baud=115200):
    """Straightforward initialization tasks.

```

```

    @param name
    An informal name of the instance. Useful if you are running
    several instances to control several serial ports at once.
    It is only used for logging output and UI messages.

```

@param path

The serial port device node living under /dev.

e.g. /dev/ttyACM0 or /dev/ttyUSB0

@param baud

The baud rate. Default is 115200 for Grbl > v0.9i.

"""

self.name = name

self.path = path

self.baud = baud

self.queue = None

self.logger = logging.getLogger('GrblStreamer.interface')

self._buf_receive = "

self._do_receive = False

def start(self, queue):

"""

Open the device node and start a Thread for reading.

@param queue

An instance of Python3's `Queue()` class.

"""

self.queue = queue

self.logger.info(

'%s: connecting to %s with baudrate %i',

self.name,

self.path,

self.baud)

self.serialport = serial.Serial(

self.path,

self.baud,

parity=serial.PARITY_NONE,

stopbits=serial.STOPBITS_ONE,

bytesize=serial.EIGHTBITS,

timeout=1,

writeTimeout=0,

)

self.serialport.flushInput()

self.serialport.flushOutput()

self._do_receive = True

self.serial_thread = threading.Thread(target=self._receiving)

self.serial_thread.start()

def stop(self):

"""

Close the device node and shut down the reading Thread.

"""

```

self._do_receive = False
self.logger.info('%s: stop()', self.name)
self.serial_thread.join()
self.logger.info('%s: JOINED thread', self.name)
self.logger.info('%s: Closing port', self.name)
self.serialport.flushInput()
self.serialport.flushOutput()
self.serialport.close()

```

```

def write(self, data):
    """
    Write `data` to the device node.
    If data is empty, no write is performed.
    The number of written characters is returned.
    """
    if len(data) > 0:
        num_written = self.serialport.write(bytes(data, 'ascii'))
        return num_written
    else:
        self.logger.debug('%s: nothing to write', self.name)

```

```

def _receiving(self):
    while self._do_receive:
        data = self.serialport.read(1)
        waiting = self.serialport.inWaiting()
        data += self.serialport.read(waiting)
        self._handle_data(data)

```

```

def _handle_data(self, data):
    try:
        asci = data.decode('ascii')
    except UnicodeDecodeError:
        self.logger.info(
            '%s: Received a non-ascii byte. Probably junk. Dropping it.',
            self.name)
        asci = ""

```

```

for i in range(0, len(asci)):
    char = asci[i]
    self._buf_receive += char
    # not all received lines are complete (end with \n)
    if char == "\n":
        self.queue.put(self._buf_receive.strip())
        self._buf_receive = ""

```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoil/.venv/Lib/site-packages/gcode_machine

"""

gcode-machine - CNC controller state machine for parsing and pre-processing of G-code
Copyright (C) 2016, 2023 Michael Franzl

This file is part of gcode-machine.

gcode-machine is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

gcode-machine is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import re
import logging
import math
```

```
class GcodeMachine:
```

```
    """ This implements a simple CNC state machine that can be used
    for simulation and processing of G-code.
```

```
    For usage, see README.md.
```

```
    Callbacks:
```

```
    on_feed_change
    : Emitted when a F keyword is parsed from the G-Code.
    : One argument: the feed rate in mm/min
```

```
    on_var_undefined
    : Emitted when a variable is to be substituted
    : but no substitution value has been set previously.
    : One argument: the key of the undefined variable
```

```
    """
```

```
    def __init__(
        self,
        impos=(0, 0, 0),
        ics="G54",
        cs_offsets={"G54": (0, 0, 0)}
    ):
```

```
);  
""" Initialization.
```

Parameters

```
-----
```

```
impos : 3-tuple  
    Initial machine position
```

```
ics : str  
    Initial coordinate system. Default: 'G54'
```

```
cs_offsets : dict  
    Default: {'G54': (0, 0, 0)}  
"""
```

```
self.logger = logging.getLogger('gcode_machine')
```

```
# @var line  
# Holds the current Gcode line  
self.line = ""
```

```
# @var vars  
# A dict holding values for variable substitution.  
self.vars = {}
```

```
# @var callback method for parent applications  
self.callback = self.__default_callback
```

```
# @var do_feed_override  
# If set to True, F commands will be replaced with the feed  
# speed set in `request_feed`. If set to False, no feed speed  
# processing will be done.  
self.do_feed_override = False
```

```
# @var do_fractionize_lines  
# If set to True, linear movements over the threshold of  
# `fract_linear_threshold` will be broken down into small line  
# segments of length `fract_linear_segment_len`. If set to False  
# no processing is done on lines.  
self.do_fractionize_lines = False
```

```
# @var do_fractionize_arcs  
# If set to True, arcs will be broken up into small line segments.  
# If set to False, no processing is done on arcs.  
self.do_fractionize_arcs = False
```

```
# @var fract_linear_threshold  
# The threshold for the fractionization of lines.  
self.fract_linear_threshold = 1
```

```

# @var fract_linear_segment_len
# The length of the segments of fractionized lines.
self.fract_linear_segment_len = 0.5

# @var spindle_factor
# Scale S by this value
self.spindle_factor = 1

# @var request_feed
# If `do_feed_override` is True, a F command will be inserted
# to the currently active command with this value.
self.request_feed = None

# @var current_feed
# The current feed rate of the machine
self.current_feed = None

# @var contains_feed
# True if the current line contains a F command
self.contains_feed = False

# @var current_distance_mode
# Contains the current distance mode of the machine as string
self.current_distance_mode = "G90"

# @var current_motion_mode
# Contains the current motion mode of the machine as integer (0 to 3)
self.current_motion_mode = 0

# @var current_plane_mode
# Contains the current plane mode of the machine as string
self.current_plane_mode = "G17"

# @var cs_offsets
# Coordinate system offsets. A Python dict with
# 3-tuples as offsets.
self.cs_offsets = cs_offsets

# @var cs
# Current coordinate system as string.
self.cs = ics

# @var pos_m
# Contains the current machine position before execution
# of the currently set line (the target of the last command)
self.pos_m = list(impos) # initial machine position

# @var pos_w
# Contains the current working position before execution
# of the currently set line (the target of the last command)
self.pos_w = [

```

```

        self.pos_m[0] - self.cs_offsets[self.cs][0],
        self.pos_m[1] - self.cs_offsets[self.cs][1],
        self.pos_m[2] - self.cs_offsets[self.cs][2]
    ]

    # @var target_m
    # Contains the position target of the currently set command in
    # the machine coordinate system.
    self.target_m = [None, None, None]

    # @var target_w
    # Contains the position target of the currently set command in
    # the currently selected coordinate system.
    self.target_w = [None, None, None]

    # @var offset
    # Contains the offset of the arc center from current position
    self.offset = [0, 0, 0]

    # @var radius
    # Contains the radius of the current arc
    self.radius = None

    # @var contains_radius
    # True if the current line contains a R word
    self.contains_radius = False

    # @var current_spindle_speed
    # Contains the current spindle speed (S word)
    self.current_spindle_speed = None

    # @var contains_spindle
    # True if the current line contains the S word
    self.contains_spindle = False

    # @var dist
    # Distance that current line will travel
    self.dist = 0

    # @var dists
    # Distance that current line will travel, in [x,y,z] directions
    self.dist_xyz = [0, 0, 0]

    # @var line_is_unsupported_cmd
    # True if current line is not in the whitelist
    self.line_is_unsupported_cmd = False

    self.comment = ""

    self.special_comment_prefix = "_gcm"

```

```

# precompile regular expressions
self._axes_regexps = []
self._axes_words = ["X", "Y", "Z"]
for i in range(0, 3):
    word = self._axes_words[i]
    self._axes_regexps.append(re.compile('.*' + word + r'([-.\d]+)'))

self._offset_regexps = []
self._offset_words = ["I", "J", "K"]
for i in range(0, 3):
    word = self._offset_words[i]
    self._offset_regexps.append(re.compile('.*' + word + r'([-.\d]+)'))

self._re_radius = re.compile(r'.*R([-.\d]+)')

self._re_use_var = re.compile(r'#(\d*)')
self._re_set_var = re.compile(r'^s#(\d+)([.\d-]+)')

self._re_feed = re.compile(r'.*F([-.\d]+)')
self._re_feed_replace = re.compile(r'F[.\d]+')

self._re_spindle = re.compile(r'.*S([.\d]+)')
self._re_spindle_replace = re.compile(r'S[.\d]+')

self._re_motion_mode = re.compile('G([0123])([^\d]|$)')
self._re_distance_mode = re.compile(r'(G9[01])([^\d]|$)')
self._re_plane_mode = re.compile(r'(G1[789])([^\d]|$)')
self._re_cs = re.compile(r'(G5[4-9])')

self._re_comment_paren_convert = re.compile(r'(\.*)((.*?))\s*$')
self._re_comment_paren_replace = re.compile(r'\(.*?\)')
self._re_comment_get_comment = re.compile(r'(\.*)((.*?))')

self._re_match_cmd_number = re.compile(r'([GMT])(\d+)')
self._re_expand_multicommands = re.compile(r'([GMT])')

# @var whitelist_commands
# Strip line from everything that is not in this list
self.whitelist_commands = {
    "G": [
        # non-modal commands
        4, # Dwell
        10, # set coordinate system
        28, # Go to predefined position
        30, # Go to predefined position
        53, # move in machine coordinates
        92, # set coordinate system offset

        # motion modes
        0, # fast linear
        1, # slow linear
    ]
}

```

2, # slow arc CW
3, # slow arc CCW
38, # probe
80, # canned cycle stop

feed rate modes

93, # inverse
94, # normal

units

20, # inch
21, # mm

distance modes

90, # absolute
91, # incremental

plane select

17, # XY
18, # ZY
19, # YZ

tool length offset

43, # set compensation
49, # cancel compensation

coordinate systems

54,
55,
56,
57,
58,
59,

],

"M": [

program flow

0,
1,
2,
30,

coolant

8,
9,

spindle

3,
4,
5,

]

```
}
```

```
def reset(self):
```

```
    """
```

```
    Reset to initial state.
```

```
    """
```

```
    self.vars = {}
```

```
    self.current_feed = None
```

```
    self.current_motion_mode = 0
```

```
    self.current_distance_mode = "G90"
```

```
    self.callback("on_feed_change", self.current_feed)
```

```
@property
```

```
def position_m(self):
```

```
    return list(self.pos_m)
```

```
@position_m.setter
```

```
def position_m(self, pos):
```

```
    self.pos_m = list(pos)
```

```
    offset = self.cs_offsets[self.cs]
```

```
    self.pos_w = [
```

```
        self.pos_m[0] - offset[0],
```

```
        self.pos_m[1] - offset[1],
```

```
        self.pos_m[2] - offset[2]
```

```
    ]
```

```
@property
```

```
def current_cs(self):
```

```
    return self.cs
```

```
@current_cs.setter
```

```
def current_cs(self, label):
```

```
    self.cs = label
```

```
    offset = self.cs_offsets[self.cs]
```

```
    self.pos_w = [
```

```
        self.pos_m[0] - offset[0],
```

```
        self.pos_m[1] - offset[1],
```

```
        self.pos_m[2] - offset[2]
```

```
    ]
```

```
def set_line(self, line):
```

```
    """
```

```
    Load a Gcode line into the machine. It will be available in `self.line`
```

```
Parameters
```

```
-----
```

```
line : str
```

```
    A single G-Code line
```

```
    """
```

```
self.line = line
```

```
self.transform_comments()
```

```
def split_lines(self):
```

```
    """
```

Some Gcode generating software spit out 'composite' commands like "M3 T2". This really is bad practice.

This method splits such 'composite' commands up into separate commands and returns a list of commands. Machine state is not changed.

```
    """
```

```
    commands = re.sub(self._re_expand_multicommands,
                       r'\n\g<0>', self.line).strip()
```

```
    lines = commands.split("\n")
```

```
    lines[0] = lines[0] + self.comment # preserve comment
```

```
    return lines
```

```
def strip(self):
```

```
    """
```

Remove blank spaces and newlines from beginning and end, and remove blank spaces from the middle of the line.

```
    """
```

```
    self.line = self.line.replace(" ", "")
```

```
    self.line = self.line.strip()
```

```
def tidy(self):
```

```
    """
```

Strips G-Code not contained in the whitelist.

```
    """
```

```
# transform [MG]\d to G\d\d for better parsing
```

```
def format_cmd_number(matchobj):
```

```
    cmd = matchobj.group(1)
```

```
    cmd_nr = int(matchobj.group(2))
```

```
    self.line_is_unsupported_cmd = not (
```

```
        cmd in self.whitelist_commands
```

```
        and cmd_nr in self.whitelist_commands[cmd]
```

```
    )
```

```
    return '{}{:02d}'.format(cmd, cmd_nr)
```

```
self.line = re.sub(self._re_match_cmd_number,
```

```
                  format_cmd_number, self.line)
```

```
if self.line_is_unsupported_cmd:
```

```
    self.line = ';' + self.line + \
```

```
        ';' + self.special_comment_prefix + \
```

```
        '.unsupported'
```

```
def parse_state(self):
```

```
    """
```

This method...

- * parses motion mode
- * parses distance mode
- * parses plane mode
- * parses feed rate
- * parses spindle speed
- * parses arcs (offsets and radii)
- * calculates travel distances

... and updates the machine's state accordingly.

"""

parse G0 .. G3 and remember

m = re.match(self._re_motion_mode, self.line)

if m:

self.current_motion_mode = int(m.group(1))

parse G90 and G91 and remember

m = re.match(self._re_distance_mode, self.line)

if m:

self.current_distance_mode = m.group(1)

parse G17, G18 and G19 and remember

m = re.match(self._re_plane_mode, self.line)

if m:

self.current_plane_mode = m.group(1)

m = re.match(self._re_cs, self.line)

if m:

self.current_cs = m.group(1)

see if current line has F

m = re.match(self._re_feed, self.line)

self.contains_feed = True if m else False

if m:

self.feed_in_current_line = float(m.group(1))

look for spindle S

m = re.match(self._re_spindle, self.line)

self.contains_spindle = True if m else False

if m:

self.current_spindle_speed = int(m.group(1))

arc parsing and calculations

if self.current_motion_mode == 2 or self.current_motion_mode == 3:

self.offset = [None, None, None]

for i in range(0, 3):

loop over I, J, K offsets

regex = self._offset_regexps[i]

m = re.match(regex, self.line)

```

        if m:
            self.offset[i] = float(m.group(1))

# parses arcs
m = re.match(self._re_radius, self.line)
self.contains_radius = True if m else False
if m:
    self.radius = float(m.group(1))

# calculate distance traveled by this G-Code cmd in xyz
self.dist_xyz = [0, 0, 0]
for i in range(0, 3):
    # loop over X, Y, Z axes
    regexp = self._axes_regexp[i]

    m = re.match(regexp, self.line)
    if m:
        if self.current_distance_mode == "G90":
            # absolute distances
            self.target_m[i] = self.cs_offsets[self.cs][i] + \
                float(m.group(1))
            self.target_w[i] = float(m.group(1))

            # calculate distance
            self.dist_xyz[i] = self.target_m[i] - self.pos_m[i]
        else:
            # G91 relative distances
            self.dist_xyz[i] = float(m.group(1))
            self.target_m[i] += self.dist_xyz[i]
            self.target_w[i] += self.dist_xyz[i]
    else:
        # no movement along this axis, stays the same
        self.target_m[i] = self.pos_m[i]
        self.target_w[i] = self.pos_w[i]

# calculate travelling distance
self.dist = math.sqrt(
    self.dist_xyz[0] * self.dist_xyz[0] +
    self.dist_xyz[1] * self.dist_xyz[1] +
    self.dist_xyz[2] * self.dist_xyz[2]
)

```

```
def fractionize(self):
```

```
    """
```

Breaks lines longer than a certain threshold into shorter segments.

Also breaks circles into segments.

Returns a list of command strings. Does not update the machine state.

```
    """
```

```
result = []
```

```
if (self.do_fractionize_lines
    and self.current_motion_mode == 1
    and self.dist > self.fract_linear_threshold):
    result = self.__fractionize_linear_motion()
```

```
elif (self.do_fractionize_arcs and
      (self.current_motion_mode == 2 or self.current_motion_mode == 3)
      ):
    result = self.__fractionize_circular_motion()
```

```
else:
    # this motion cannot be fractionized
    # return the line as it was passed in
    result = [self.line]
```

```
result[0] = result[0] + self.comment # preserve comment
return result
```

```
def done(self):
```

```
    """
```

```
    Call this method when all processing/inspecting of a command
    has been done.
```

```
    This will virtually 'move' the tool of the machine
    if the current command is a motion command.
```

```
    """
```

```
if not (self.current_motion_mode == 0 or self.current_motion_mode == 1):
    # only G0 and G1 can stay active without re-specifying
    self.current_motion_mode = None
```

```
# move the 'tool'
```

```
for i in range(0, 3):
```

```
    # loop over X, Y, Z axes
```

```
    if self.target_m[i] is not None: # keep state
```

```
        self.pos_m[i] = self.target_m[i]
```

```
        self.pos_w[i] = self.target_w[i]
```

```
# re-add comment
```

```
self.line += self.comment
```

```
def find_vars(self):
```

```
    """
```

```
    Parses all variables in a G-Code line (#1, #2, etc.) and populates
    the internal `vars` dict with corresponding keys and values
```

```
    """
```

```
m = re.match(self._re_set_var, self.line)
```

```
if m:
```

```
    key = m.group(1)
```

```
    val = str(float(m.group(2))) # get rid of extra zeros
```

```
self.vars[key] = val
self.line = ";" + self.line
```

```
# find variable usages
keys = re.findall(self._re_use_var, self.line)
for key in keys:
    if key not in self.vars:
        self.vars[key] = None
```

```
def substitute_vars(self):
    """
    Substitute a variable with a value from the `vars` dict.
    """
    keys = re.findall(self._re_use_var, self.line)

    for key in keys:
        val = None
        if key in self.vars:
            val = self.vars[key]

        if val is None:
            self.line = ""
            self.callback("on_var_undefined", key)
            return self.line
        else:
            self.line = self.line.replace("#" + key, str(val))
            self.logger.info("substituted variable #{0} -> {1}".format(key, val))
```

```
def scale_spindle(self):
    if self.contains_spindle:
        # strip the original S setting
        self.line = re.sub(self._re_spindle_replace, "", self.line).strip()
        self.line += "S{:d}".format(
            int(self.current_spindle_speed * self.spindle_factor))
```

```
def override_feed(self):
    """
    Call this method to

    * get a callback when the current command contains an F word
    *
    """
```

```
if not self.do_feed_override and self.contains_feed:
    # Notify caller of detected feed in current line (useful for UIs)
    if self.current_feed != self.feed_in_current_line:
        self.callback("on_feed_change", self.feed_in_current_line)
        self.current_feed = self.feed_in_current_line
```

```
if self.do_feed_override and self.request_feed:
```

```

if self.contains_feed:
    # strip the original F setting
    self.logger.info("stripping feed " + self.line)
    self.line = re.sub(self._re_feed_replace,
        "", self.line).strip()

if (self.current_feed != self.request_feed):
    self.line += "F{:0.1f}".format(self.request_feed)
    self.current_feed = self.request_feed
    self.logger.info("overriding feed " + str(self.current_feed))
    self.callback("on_feed_change", self.current_feed)

# TODO: This method can be made 'private' as it is called from `set_line()`
def transform_comments(self):
    """
    Comments in Gcode can be set with semicolon or parentheses.
    This method transforms parentheses comments to semicolon comments.
    """

    # transform () comment at end of line into semicolon comment
    self.line = re.sub(self._re_comment_paren_convert,
        r"\g<1>;\g<2>", self.line)

    # remove all in-line () comments
    self.line = re.sub(self._re_comment_paren_replace, "", self.line)

    m = re.match(self._re_comment_get_comment, self.line)
    if m:
        self.line = m.group(1)
        self.comment = m.group(2)
    else:
        self.comment = ""

# 'private' methods

def __fractionize_circular_motion(self):
    """
    This function is a direct port of Grbl's C code into Python (gcode.c)
    with slight refactoring for Python by Michael Franzl.
    See https://github.com/grbl/grbl
    """

    # implies self.current_motion_mode == 2 or self.current_motion_mode == 3

    if self.current_plane_mode == 'G17':
        axis_0 = 0 # X axis
        axis_1 = 1 # Y axis
        axis_linear = 2 # Z axis
    elif self.current_plane_mode == 'G18':
        axis_0 = 2 # Z axis
        axis_1 = 0 # X axis

```

```

    axis_linear = 1 # Y axis
elif self.current_plane_mode == 'G19':
    axis_0 = 1 # Y axis
    axis_1 = 2 # Z axis
    axis_linear = 0 # X axis

is_clockwise_arc = True if self.current_motion_mode == 2 else False

# deltas between target and (current) position
x = self.target_w[axis_0] - self.pos_w[axis_0]
y = self.target_w[axis_1] - self.pos_w[axis_1]

if self.contains_radius:
    # RADIUS MODE
    # R given, no IJK given, self.offset must be calculated

    if tuple(self.target_w) == tuple(self.pos_w):
        self.logger.error(
            "Arc in Radius Mode: Identical start/end {}".format(self.line))
        return [self.line]

    h_x2_div_d = 4.0 * self.radius * self.radius - x * x - y * y

    if h_x2_div_d < 0:
        self.logger.error(
            "Arc in Radius Mode: Radius error {}".format(self.line))
        return [self.line]

    # Finish computing h_x2_div_d.
    h_x2_div_d = -math.sqrt(h_x2_div_d) / math.sqrt(x * x + y * y)

    if not is_clockwise_arc:
        h_x2_div_d = -h_x2_div_d

    if self.radius < 0:
        h_x2_div_d = -h_x2_div_d
        self.radius = -self.radius

    self.offset[axis_0] = 0.5*(x-(y*h_x2_div_d))
    self.offset[axis_1] = 0.5*(y+(x*h_x2_div_d))

else:
    # CENTER OFFSET MODE, no R given so must be calculated

    if self.offset[axis_0] is None or self.offset[axis_1] is None:
        raise Exception(
            "Arc in Offset Mode: No offsets in plane. {}".format(self.line))
        # self.logger.error("Arc in Offset Mode: No offsets in plane")
        # return [self.line]

# Arc radius from center to target

```

```

x -= self.offset[axis_0]
y -= self.offset[axis_1]
target_r = math.sqrt(x * x + y * y)

```

```

# Compute arc radius for mc_arc.
# Defined from current location to center.
self.radius = math.sqrt(
    self.offset[axis_0] * self.offset[axis_0] +
    self.offset[axis_1] * self.offset[axis_1]
)

```

```

# Compute difference between current location and target radii for final error-checks.
delta_r = math.fabs(target_r - self.radius)
if delta_r > 0.005:
    if delta_r > 0.5:
        raise Exception("Arc in Offset Mode: Invalid Target. r={:f} delta_r={:f} {}".format(
            self.radius, delta_r, self.line))

    if delta_r > (0.001 * self.radius):
        raise Exception("Arc in Offset Mode: Invalid Target. r={:f} delta_r={:f} {}".format(
            self.radius, delta_r, self.line))

```

```

gcode_list = self.__mc_arc(
    self.pos_w,
    self.target_w,
    self.offset,
    self.radius,
    axis_0,
    axis_1,
    axis_linear,
    is_clockwise_arc
)
return gcode_list

```

```

def __mc_arc(
    self,
    position,
    target,
    offset,
    radius,
    axis_0,
    axis_1,
    axis_linear,
    is_clockwise_arc
):
    """

```

This function is a direct port of Grbl's C code into Python (motion_control.c) with slight refactoring for Python by Michael Franzl.
 See <https://github.com/grbl/grbl>

```

gcode_list = []
gcode_list.append("; " + self.special_comment_prefix +
    ".arc_begin[{}]" .format(self.line))

do_restore_distance_mode = False
if self.current_distance_mode == "G91":
    # it's bad to concatenate many small floating point segments due to accumulating errors
    # each arc will use G90
    do_restore_distance_mode = True
    gcode_list.append("G90")

center_axis0 = position[axis_0] + offset[axis_0]
center_axis1 = position[axis_1] + offset[axis_1]
# radius vector from center to current location
r_axis0 = -offset[axis_0]
r_axis1 = -offset[axis_1]
# radius vector from target to center
rt_axis0 = target[axis_0] - center_axis0
rt_axis1 = target[axis_1] - center_axis1

angular_travel = math.atan2(
    r_axis0 * rt_axis1 - r_axis1 * rt_axis0,
    r_axis0 * rt_axis0 + r_axis1 * rt_axis1
)

arc_tolerance = 0.004
arc_angular_travel_epsilon = 0.0000005

if is_clockwise_arc: # Correct atan2 output per direction
    if angular_travel >= -arc_angular_travel_epsilon:
        angular_travel -= 2*math.pi
    else:
        if angular_travel <= arc_angular_travel_epsilon:
            angular_travel += 2*math.pi

segments = math.floor(
    math.fabs(0.5 * angular_travel * radius) /
    math.sqrt(arc_tolerance * (2 * radius - arc_tolerance))
)

words = ["X", "Y", "Z"]
if segments:
    theta_per_segment = angular_travel / segments
    linear_per_segment = (
        target[axis_linear] - position[axis_linear]) / segments

    position_last = list(position)
    for i in range(1, segments):
        cos_Ti = math.cos(i * theta_per_segment)
        sin_Ti = math.sin(i * theta_per_segment)
        r_axis0 = -offset[axis_0] * cos_Ti + offset[axis_1] * sin_Ti

```

```

r_axis1 = -offset[axis_0] * sin_Ti - offset[axis_1] * cos_Ti

position[axis_0] = center_axis0 + r_axis0
position[axis_1] = center_axis1 + r_axis1
position[axis_linear] += linear_per_segment

gcodeline = ""
if i == 1:
    gcodeline += "G1"

for a in range(0, 3):
    if position[a] != position_last[a]: # only write changes
        txt = "{}{:0.3f}".format(words[a], position[a])
        txt = txt.rstrip("0").rstrip(".")
        gcodeline += txt
        position_last[a] = position[a]

if i == 1:
    if self.contains_feed:
        gcodeline += "F{:1f}".format(
            self.feed_in_current_line)
    if self.contains_spindle:
        gcodeline += "S{:d}".format(self.current_spindle_speed)

gcode_list.append(gcodeline)

# make sure we arrive at target
gcodeline = ""
if segments <= 1:
    gcodeline += "G1"

for a in range(0, 3):
    if target[a] != position[a]:
        txt = "{}{:0.3f}".format(words[a], target[a])
        txt = txt.rstrip("0").rstrip(".")
        gcodeline += txt

if segments <= 1:
    # no segments were rendered (very small arc)
    # so we have to put S and F here
    if self.contains_feed:
        gcodeline += "F{:1f}".format(self.feed_in_current_line)
    if self.contains_spindle:
        gcodeline += "S{:d}".format(self.current_spindle_speed)

gcode_list.append(gcodeline)

if do_restore_distance_mode:
    gcode_list.append(self.current_distance_mode)

gcode_list.append("; " + self.special_comment_prefix + ".arc_end")

```

```

return gcode_list

def __fractionize_linear_motion(self):
    gcode_list = []
    gcode_list.append("; " + self.special_comment_prefix +
        ".line_begin[{}].format(self.line))

    num_fractions = int(self.dist / self.fract_linear_segment_len)

    for k in range(0, num_fractions):
        # render segments
        txt = ""
        if k == 0:
            txt += "G1"

        for i in range(0, 3):
            # loop over X, Y, Z axes
            segment_length = self.dist_xyz[i] / num_fractions
            coord_rel = (k + 1) * segment_length
            if self.current_distance_mode == "G90":
                # absolute distances
                coord_abs = self.pos_w[i] + coord_rel
                if coord_rel != 0:
                    # only output for changes
                    txt += "{}{:0.3f}".format(
                        self._axes_words[i], coord_abs)
                    txt = txt.rstrip("0").rstrip(".")
            else:
                # relative distances
                txt += "{}{:0.3f}".format(
                    self._axes_words[i], segment_length)
                txt = txt.rstrip("0").rstrip(".")

        if k == 0:
            if self.contains_feed:
                txt += "F{:0.1f}".format(self.feed_in_current_line)
            if self.contains_spindle:
                txt += "S{:d}".format(self.current_spindle_speed)

        gcode_list.append(txt)

    gcode_list.append("; " + self.special_comment_prefix + ".line_end")
    return gcode_list

def __default_callback(self, status, *args):
    self.logger.info("default callback", status, args)

```